



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Triennale in Informatica

Progetto Ingegneria del Software

BugBoard26

Anno Accademico 2025/2026

Candidato
Marta Pagliuso
matr. N86004844

Indice

1	Introduzione	4
2	Documento dei Requisiti Software	6
2.1	Glossario	6
2.1.1	Termini del dominio applicativo	6
2.1.2	Termini tecnici e metodologici	8
2.2	Modellazione dei Casi d'Uso	9
2.2.1	Individuazione dei casi d'uso	9
2.2.2	Diagrammi dei casi d'uso	11
2.3	Individuazione e caratterizzazione del target degli utenti	12
2.3.1	User personas	13
2.4	Descrizione dei requisiti non funzionali e di dominio	15
2.4.1	Requisiti non funzionali	15
2.4.2	Requisiti di dominio	17
2.5	Descrizione Testuale Strutturata dei Casi d'Uso	17
2.5.1	Cockburn: Segnala una nuova issue	18
2.5.2	Cockburn: Assegna una issue e risoluzione	21
2.6	Prototipazione visuale via Mock-up	23
3	Documento di Design del Sistema	26
3.1	Progettazione architetturale e scelte di design	26
3.1.1	Il Server	27
3.1.2	Il Client	28
3.2	Scelte tecnologiche adottate	29
3.2.1	Linguaggio e ambiente di esecuzione	29
3.2.2	Framework di back-end	30

3.2.3	Persistenza dei dati	30
3.2.4	Sicurezza e autenticazione	31
3.2.5	Validazione dei dati	31
3.2.6	Gestione delle immagini	32
3.2.7	Notifiche per posta elettronica	32
3.2.8	Framework di front-end	32
3.2.9	Strumenti di sviluppo e verifica	32
3.3	Diagramma per la persistenza dei dati	33
3.3.1	Descrizione del diagramma per la persistenza dei dati	34
3.4	Scelte di design dell'interfaccia utente adottate	36
3.4.1	La palette cromatica	37
3.4.2	Coerenza visiva tra le viste	42
3.4.3	Prevenzione dell'errore	42
3.4.4	Assenza in luogo della disabilitazione	43
3.4.5	Riscontro sullo stato del sistema	44
3.4.6	Efficienza per l'uso ripetuto	44
3.5	Diagramma delle classi di design	45
4	Processo di sviluppo	49
4.1	Gestione del versioning	49
4.2	Report di qualità del codice	50
4.2.1	Stato rilevato dall'analisi iniziale	51
4.2.2	Anomalie rilevate e interventi adottati	51
4.2.3	Stato conseguente agli interventi	52
5	Testing e valutazione dell'usabilità	54
5.1	Testing	54
5.1.1	Test plan	54
5.1.2	Strategie di progettazione dei test	58
5.1.3	Test di unità automatici	60
5.1.4	Risultati e difetti individuati	65
5.2	Valutazione dell'usabilità	68
5.2.1	Expert review	68
5.2.2	Test con utenti reali	72

Capitolo 1

Introduzione

BugBoard26 è una piattaforma per la gestione collaborativa delle segnalazioni relative a progetti software. Il sistema permette agli utenti di segnalare issues, seguirne lo stato, assegnare ad un responsabile la risoluzione ed attribuire il lavoro svolto. È uno strumento interno, per un contesto aziendale, dove la tracciabilità delle issues e la chiarezza delle responsabilità sono requisiti essenziali.

L'elemento cardine del sistema è quello delle segnalazioni. Qualsiasi utente autenticato può creare una issue indicando il titolo e descrizione, e classificandolo a seconda che si tratti di un guasto, una richiesta di chiarimenti, una carenza di documentazione o di una proposta di miglitoria. Al segnalatore viene data la possibilità di indicare una priorità e di allegare un'immagine che evidenzi il problema; entrambi i dati sono però opzionali, in quanto non sempre chi rileva un difetto è in grado di valutarne l'urgenza. Gli amministratori si occupano della gestione dei compiti, assegnando una issue a un altro utente, che sarà responsabile della sua risoluzione. Al completamento della issue, il sistema provvede a informare chi l'aveva sollevata, sia tramite una notifica interna all'applicazione sia attraverso un messaggio di posta elettronica, in modo che il segnalatore non debba verificarne di volta in volta lo stato.

Attorno a ciascuna segnalazione si sviluppa una conversazione, avviata tramite l'inserimento di commenti e che consente a chiunque abbia accesso al sistema di chiedere chiarimenti, riferire tentativi o documentare l'attività svolta. Poiché il numero di segnalazioni tende ad aumentare nel tempo, il sistema permette agli utenti di effettuare una ricerca mirata tramite ricerca testuale che opera su titolo e descrizione o filtri per stato, tipologia e priorità, combinabili tra loro. Agli amministratori è inoltre fornita una vista di sintesi che fornisce il quadro complessivo dell'attività del sistema. In particolare è

possibile visualizzare quante segnalazioni risultano ancora aperte, quante attendono un responsabile, quanto hanno superato il termine assegnato, quale sia il carico di lavoro per ciascun utente e quanto tempo mediamente intercorra tra la segnalazione e la sua risoluzione.

Infine è necessario precisare che non è prevista alcuna forma di registrazione autonoma, ma essa è delegata ai soli amministratori. Questi hanno il compito di inserire i dati dell'utente (nome e cognome), il ruolo che gli spetta e una password temporanea. Sarà poi il sistema a generare automaticamente l'email consentirgli l'accesso.

Capitolo 2

Documento dei Requisiti Software

Il seguente capitolo mostra l'analisi e la specifica dei requisiti del sistema BugBoard26. L'obiettivo è formalizzare le funzionalità richieste dal committente, così da fornire una base chiara e solida per le successive fasi di progettazione e implementazione.

2.1 Glossario

Il glossario raccoglie i termini ricorrenti nel documento. Nella prima parte vengono definiti i termini del dominio applicativo, ovvero quelli riferiti alla gestione delle segnalazioni; nella seconda invece sono raccolti i termini tecnici e metodologici utilizzati nella descrizione delle scelte progettuali e delle attività di validazione.

2.1.1 Termini del dominio applicativo

Termine	Definizione
Issue	Segnalazione inserita da un utente autenticato.
Tipo di issue	Categoria a cui una issue appartiene. I valori ammessi sono: bug (malfunzionamento), question (chiarimento), documentation (problema relativo alla documentazione) e feature (nuova funzionalità).
Stato	Fase in cui si trova la issue. I valori ammessi sono: todo (ancora da iniziare), in progress (in lavorazione), done (risolta), closed (chiusa senza risoluzione).

Priorità	Attributo facoltativo di una issue che indica il grado di urgenza della stessa. I valori sono: bassa , media , alta , critica .
Segnalatore	Utente che ha creato la issue.
Assegnatario	Utente a cui è stata assegnata una issue.
Scadenza	Data facoltativa entro la quale la issue deve essere risolta.
Commento	Messaggio testuale associato a una issue, inserito da un utente autenticato.
Notifica	Avviso generato dal sistema e destinato all'utente segnalatore quando la issue creata è stata risolta.
Ruolo	Indica il grado di permesso che un utente possiede. Un admin (amministratore) possiede il grado massimo, dunque ha tutti i permessi; un utente può creare una issue e/o risolverla; un viewer (utente in sola lettura) può solo visualizzare le issue senza modificarle.
Amministratore di default	Account amministrativo predisposto automaticamente al primo avvio del sistema.
Password Temporanea	Password assegnata alla creazione di un account. L'utente è obbligato a modificarla al primo accesso.
Dashboard	Sezione riservata agli amministratori per la visualizzazione di informazioni aggregate sulle issue: numero di segnalazioni aperte, non assegnate e scadute, distribuzione per stato e per tipo, carico di lavoro per utente e tempo medio di risoluzione.
Tempo di risoluzione	Intervallo di tempo tra la creazione di una issue e il momento in cui è stata portata per la prima volta nello stato done .

Tabella 2.1: Termini del dominio applicativo

2.1.2 Termini tecnici e metodologici

Termine	Definizione
Back-end	Componente server del sistema responsabile della logica di business e dell'accesso ai dati. Funziona in modo indipendente dal client.
Front-end	Componente client del sistema responsabile dell'interfaccia utente.
API	Insieme di procedure che consentono la comunicazione tra diverse parti di un sistema o tra sistemi diversi.
Endpoint	Singolo punto di accesso di una API, identificato dalla combinazione di un metodo HTTP e di un percorso.
SPA (Single Page Application)	Applicazione web che carica una singola pagina e ne aggiorna il contenuto.
ORM (Object-Relational Mapping)	Strumento che consente di comunicare con un database attraverso costrutti del linguaggio di programmazione anziché istruzioni SQL.
JWT (JSON Web Token)	Token che trasporta le informazioni di identità dell'utente.
Access Token	Token di breve durata che autorizza l'accesso alle risorse protette.
Refresh Token	Token a lunga durata che consente di ottenere un nuovo Access Token senza ripetere l'autenticazione.
Cookie HttpOnly	Cookie non accessibile al codice JavaScript utilizzato per diminuire il rischio di esposizione dei token a utenti malevoli.
Hashing	Trasformazione di una stringa di lunghezza variabile in una stringa di lunghezza fissa. Viene applicata alla password per aumentarne la sicurezza.

RBAC	
(Role-Based Access Control)	Modello di controllo degli accessi in cui i permessi sono associati a ruoli, e i ruoli agli utenti.
Caso d'uso (use case)	Descrizione delle interazioni tra uno o più attori del sistema.
Attore	Entità che interagisce con il sistema. Può essere una persona o un sistema esterno.
User persona	Rappresentazione sintetica e verosimile di un utente tipo.
Mock-up	Prototipo statico dell'interfaccia grafica, impiegato per validare i contenuti e animazioni dell'interfaccia grafica.
UML (Unified Modeling Language)	Linguaggio visuale standard per progettare e documentare sistemi software.

Tabella 2.2: Termini tecnici e metodologici

2.2 Modellazione dei Casi d'Uso

Il modello dei Casi d'Uso è uno dei principali strumenti per descrivere i requisiti funzionali del sistema. Nato da Ivan Jacobson e poi entrato a far parte dello standard UML, descrive il sistema dal punto di vista da chi lo utilizza e non a partire dalla sua architettura. In particolare un caso d'uso è una sequenza di interazioni tra il sistema ed uno o più attori. Queste interazioni devono interessare il raggiungimento di un obiettivo per l'attore. Questo significa che un caso d'uso non deve riguardare ad esempio l'apertura di una connessione o validare un campo, ma gli obiettivi che l'utente si pone quando si rivolge al sistema.

2.2.1 Individuazione dei casi d'uso

Descrivere i casi d'uso ha il vantaggio di essere comprensibile sia ai tecnici che ai clienti, e quindi di costruire un terreno comune sul quale verificare correttezza e completezza dei requisiti prima che si dia avvio all'implementazione. Per questo motivo, verranno di seguito elencati gli obiettivi del sistema in formato tabellare, in modo da ottenere una schematizzazione più chiara delle informazioni.

Obiettivi
Al primo avvio, il sistema mette a disposizione un account amministrativo di default.
Un amministratore può creare ulteriori utente, indicando se si tratti di un utente normale o di amministrazione.
Un amministratore può creare utenze in sola lettura destinati a soggetti esterni all'organizzazione.
Un utente modifica la propria password al primo accesso.
Un utente può creare una issue indicando titolo, descrizione e tipologia.
Un utente può indicare la priorità di una issue che sta inserendo.
Un utente può modificare titolo, descrizione, tipologia e priorità delle proprie issues.
Chiunque acceda al sistema può visualizzare l'elenco delle segnalazioni.
Chiunque acceda al sistema può cercare segnalazioni tramite parole chiavi o filtrandole per tipologia e/o priorità.
Chiunque acceda al sistema può visualizzare il dettaglio di una segnalazione e la relativa discussione.
Un amministratore può assegnare una issue a un utente.
Un amministratore può fissare o rimuovere la scadenza di una issue.
Un amministratore può eliminare una issue.
L'utente a cui è stata affidata una segnalazione, può modificarne lo stato.
Il sistema informa l'autore di una segnalazione quando questa viene risolta.
Un utente può inserire dei commenti.
Un utente in sola lettura può visualizzare segnalazioni e commenti senza modificarli.
Un amministratore può visualizzare l'elenco degli utenti presenti nel sistema.
Un amministratore può visualizzare la dashboard del sistema.

Tabella 2.3: Obiettivi del software BugBoard26

2.2.2 Diagrammi dei casi d'uso

L'analisi fornita nella sezione precedente ha fornito tutte le informazioni necessarie per la realizzazione dei diagrammi dei casi d'uso. Il primo mostra la gestione delle segnalazioni, mentre il secondo mostra l'accesso e l'amministrazione degli account.

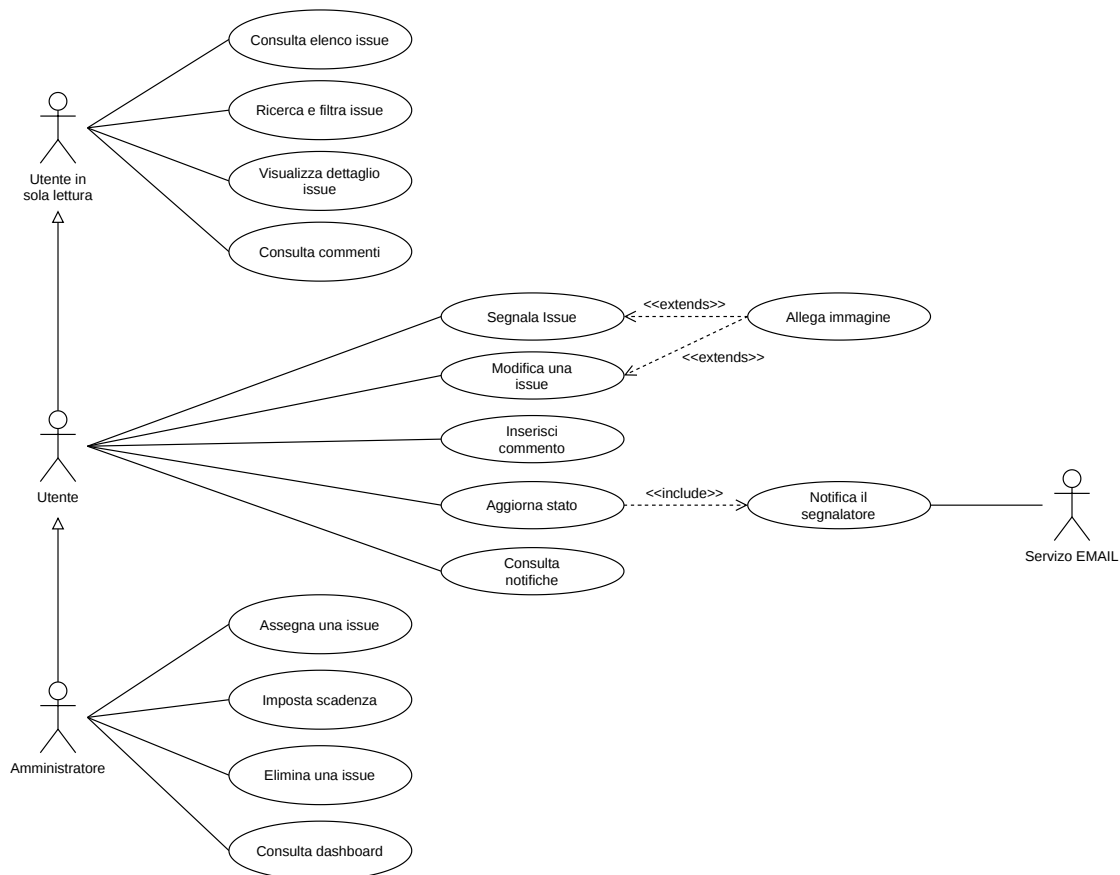


Figura 2.1: Diagramma dei Casi d'uso della Gestione delle Segnalazioni

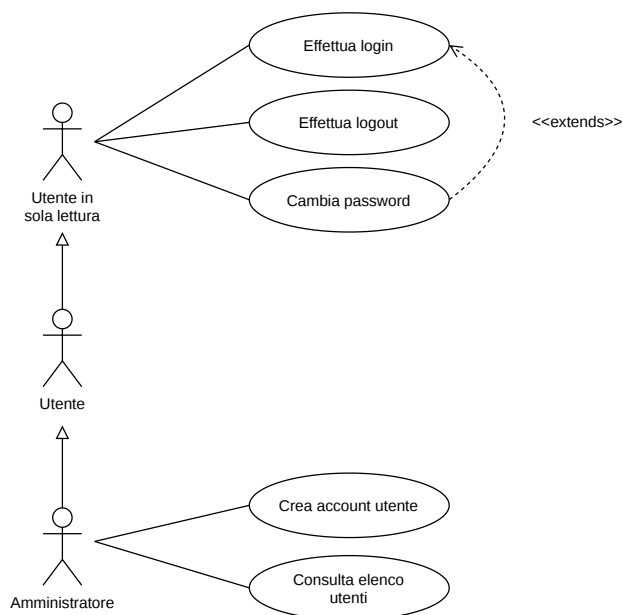


Figura 2.2: Diagramma dei Casi d'uso dell'Accesso e amministrazione degli account

2.3 Individuazione e caratterizzazione del target degli utenti

Il sistema BugBoard26 si rivolge a un contesto aziendale interno, nel quale gli utilizzatori non sono un pubblico indistinto ma un insieme circoscritto di persone che condividono un progetto e una responsabilità professionale. Gli utilizzatori si dividono in tre categorie differenti.

C'è *utente*, che costituisce l'utenza più numerosa. Fa parte del personale tecnico e già familiare con strumenti di tracciamento dei difetti. Poiché alterna momenti in cui segnala problemi rilevanti e momenti in cui aggiorna lo stato di quanto gli è stato affidati, favorisce interfacce dense di informazione, filtri combinabili e percorsi brevi. Poi c'è l'*amministratore*, il quale coordina le utenze e le segnalazioni. Il suo interesse ricade prevalentemente sul controllare quante segnalazioni sono rimaste senza responsabile, quali abbiano superato il termine e come sia distribuito il carico di lavoro tra i collaboratori. Infine è presente il *viewer* (utente in sola lettura). Si tratta di una figura esterna che non possiede alcuna competenza tecnica. Per costui è essenziale che l'interfaccia sia immediatamente comprensibile e leggibile, senza la presenza di comandi

2.3. INDIVIDUAZIONE E CARATTERIZZAZIONE DEL TARGET DEGLI UTENTI13

la cui attivazione sarebbe comunque negata.

La definizione del target utenti è un passaggio fondamentale per la progettazione di un sistema software. Comprendere a chi è destinato il prodotto consente di orientare le scelte funzionali e di interfaccia verso esigenze concrete. Lo strumento adottato per questo è scopo è quello delle *user personas*. Si tratta di rappresentazioni di utenti non reali, costruite a partire dall'analisi del contesto d'uso, al fine di valutare ciascuna scelta progettuale su esigenze riconducibili e non preferenze arbitrarie.

2.3.1 User personas

Come già accennato, le user personas sono rappresentazioni sintetiche e verosimili di utilizzatori tipo. Non si riferiscono a persone reali, ma sono comunque costruite a partire da profili individuati, utili nel valutare ciascuna scelta progettuale su esigenze riconducibili e non preferenze arbitrarie.

User Persona

Profilo	Background	Audience Insights
 Name: Chiara Mancuso Età: 41 anni Lavoro: Project manager Income range: 48.000 € annui lordi Location: Napoli	Breve bio: Ha iniziato la carriera come sviluppatrice e da sei anni ricopre un ruolo di coordinamento. Segue due team e mantiene i rapporti con i clienti dell'azienda. Stile di vita e valori: Predilige le decisioni fondate su dati verificabili anziché su impressioni. Attribuisce importanza all'equità nella distribuzione del lavoro e alla trasparenza verso i collaboratori. Contesto professionale: Tra riunioni e gestione operativa, accede al sistema 2-3 volte al giorno per sessioni prolungate. In questa fase consulta la vista aggregata, assegna le segnalazioni pendenti e fissa le scadenze per le urgenze.	Obiettivi: • Obiettivi a breve termine: ◦ Individuare issue trascurate o scadute ◦ Assegnare subito una nuova issue • Obiettivi a lungo termine: ◦ Equilibrare il carico tra i collaboratori ◦ Disporre di dati da riferire • Fattori chiave e priorità: ◦ Visione d'insieme - Equità ◦ Prevedibilità dei tempi Ostacoli: • Sfide pratiche: ◦ Elenchi dove servirebbe una sintesi • Frustrazione emotiva: ◦ Accorgersi tardi di una issue ferma ◦ Non poter giustificare i tempi con dati • Ostacoli al successo: ◦ Numeri privi di contesto ◦ Conteggi manuali che rallentano

User Persona

Profile	Background	Audience Insights
 Name: Davide Ruggiero Età: 29 anni Lavoro: Sviluppatore back-end Income range: 32.000 €/anno Location: Napoli	Breve bio: Laureato in Informatica, lavora da quattro anni presso una software house che sviluppa gestionali per aziende manifatturiere. Fa parte di un team di otto persone. Stile di vita e valori: Predilige efficienza e sobrietà degli strumenti che utilizza. Diffida delle procedure con passaggi non necessari e preferisce interfacce dense di informazione. Contesto professionale: Alterna lo sviluppo di nuove funzionalità alla correzione dei difetti. Accede al sistema più volte al giorno per verificare le assegnazioni, segnalare problemi e aggiornare lo stato di quanto risolto.	Obiettivi: • <i>Obiettivi a breve termine:</i> ◦ Trovare subito le issue assegnate ◦ Segnalare un problema in pochi istanti • <i>Obiettivi a lungo termine:</i> ◦ Documentare gli interventi svolti ◦ Ridurre il tempo speso in gestione • <i>Fattori chiave e priorità:</i> ◦ Rapidità · Autonomia ◦ Chiarezza sulle responsabilità Ostacoli: • <i>Sfide pratiche:</i> ◦ Elenchi senza filtri • <i>Frustrazione emotiva:</i> ◦ Percepire lo strumento come adempimento ◦ Timore che una issue vada persa • <i>Ostacoli al successo:</i> ◦ Tanti passaggi inutili ◦ Nessun riscontro sull'esito

User Persona

Profile	Background	Audience Insights
 Name: Paolo Sanna Età: 53 anni Lavoro: Responsabile sistemi Income range: 55.000 €/anno Location: Caserta	Breve bio: Lavora presso un'azienda manifatturiera che ha commissionato lo sviluppo di un modulo gestionale. Si occupa dei rapporti con i fornitori di software ma non partecipa alle attività di sviluppo. Stile di vita e valori: Attribuisce valore alla trasparenza nei rapporti con i fornitori. Preferisce verificare autonomamente lo stato dei lavori piuttosto che dover sollecitare aggiornamenti. Contesto professionale: Accede al sistema di rado, all'incirca una volta alla settimana, con il solo scopo di verificare se i problemi segnalati dalla propria azienda siano stati presi in carico. Non conserva memoria della struttura dell'interfaccia da un accesso all'altro.	Obiettivi: • <i>Obiettivi a breve termine:</i> ◦ Monitorare stato segnalazioni ◦ Monitorare presa in carico delle segnalazioni. • <i>Obiettivi a lungo termine:</i> ◦ Ridurre le email ◦ Valutare oggettivamente il fornitore. • <i>Fattori chiave e priorità:</i> ◦ Autonomia · immediatezza ◦ Completezza Ostacoli: • <i>Sfide pratiche:</i> ◦ Terminologia tecnica non spiegata ◦ Inesperienza con strumenti • <i>Frustrazione emotiva:</i> ◦ Comandi inattivi scambiati per bug ◦ Timore di informazioni nascoste. • <i>Ostacoli al successo:</i> ◦ Interfacce pensate per utenti quotidiani

2.4 Descrizione dei requisiti non funzionali e di dominio

I requisiti non funzionali sono le caratteristiche che un sistema deve avere, a prescindere dalle funzionalità specifiche offerte. Non si tratta di operazioni che l'utente può eseguire, ma di vincoli su come queste operazioni devono essere realizzate, come, ad esempio, il livello di protezione delle informazioni, i tempi di risposta attesi, la facilità d'uso, e altro ancora. La loro assenza potrebbe rendere il prodotto inadeguato, anche se tutte le funzionalità richieste sono presenti. Immaginiamo di avere un sistema che memorizza le password in chiaro. Soddisfa il requisito di autenticazione, ma risulterebbe inutilizzabile in un contesto aziendale.

Accanto a questi, ci sono i requisiti di dominio, che derivano dalla specificità del contesto applicativo piuttosto che da scelte progettuali. Questi requisiti vincolano il sistema, poiché riflettono regole proprie dell'ambito in cui opera, e la loro violazione potrebbe portare a comportamenti che non corrispondono alle aspettative degli utenti.

2.4.1 Requisiti non funzionali

- Sicurezza

ID	Requisito
RNF01	Le password non devono essere memorizzate in chiaro, ma sotto forma di hash prodotto da una funzione resistente ad attacchi.
RNF02	L'accesso a qualsiasi funzionalità diversa dall'autenticazione deve dipendere dal possesso di credenziali valide.
RNF03	I token di sessione non devono essere accessibili al codice eseguito dal browser.
RNF04	Ogni operazione deve essere autorizzata dal server, indipendentemente dai controlli fatti dall'interfaccia.
RNF05	Gli account con password temporanea devono modificare la stessa prima di effettuare qualsiasi altra operazione.
RNF06	I messaggi di errore devono permettere di distinguere un'utente inesistente da una password errata.

- **Usabilità**

ID	Requisito
RNF07	Le funzionalità non accessibili agli utenti che non ne posseggono i permessi non devono essere presentate.
RNF08	I comandi che avviano un'operazione devono essere disattivati fino a quando non vengono inseriti tutti i dati richiesti.
RNF09	Il sistema deve segnalare lo stato delle operazioni concluse e quelli in corso.
RNF10	Il contrasto tra testo e sfondo deve rispettare il livello AA delle Web Content Accessibility Guidelines.

- **Prestazioni ed efficienza**

ID	Requisito
RNF11	La consultazione dell'elenco delle segnalazione deve restituire i risultati in modo paginato.
RNF12	Le informazioni aggregate della dashboard devono essere calcolate dal sistema di gestione dei dati e non elaborate a valle.
RNF13	Le immagini allegate devono essere ridimensionate e compresse prima della memorizzazione.
RNF14	La ricerca testuale non deve generare una richiesta al server per ciascun carattere digitato

- **Affidabilità**

ID	Requisito
RNF15	Il fallimento dell'invio di una notifica non deve impedire il completamento dell'operazione che l'ha generata.
RNF16	Il sistema deve verificare la validità dei dati ricevuti prima di utilizzarli.

- **Manutenibilità e portabilità**

ID	Requisito
RNF17	Le componenti back-end e front-end devono essere totalmente indipendenti l'una dall'altra.
RNF18	I parametri di configurazione dipendenti dall'ambiente di esecuzione devono essere esterni al codice sorgente.
RNF19	La logica applicativa deve essere verificabile anche senza le risorse esterne.

2.4.2 Requisiti di dominio

ID	Requisito
RD01	Gli indirizzi di posta elettronica degli utenti appartengono al dominio aziendale e seguono la struttura <code>nome.cognome</code> .
RD02	Non è prevista alcuna forma di registrazione autonoma.
RD03	Il sistema è provvisto di un account di amministrazione di default.
RD04	Ogni segnalazione ha un unico creatore e al più un unico responsabile.
RD05	Solo il responsabile della segnalazione può modificarne lo stato.
RD06	Gli utenti in sola lettura non possono ricevere segnalazioni da risolvere.
RD07	Il tempo di risoluzione viene calcolato dal momento della creazione alla prima dichiarazione di risoluzione.
RD08	L'eliminazione di una segnalazione comporta l'eliminazione dei commenti e delle notifiche associate.

2.5 Descrizione Testuale Strutturata dei Casi d'Uso

I diagrammi dei casi d'uso, per loro natura sintetica, mostrano le funzionalità del sistema e gli attori coinvolti, ma non ne descrivono il funzionamento dettagliato. È dunque fondamentale fornire una descrizione testuale che chiarisca la sequenza delle interazioni, le condizioni necessarie per l'avvio e gli esiti attesi. A questo scopo è stata utilizzata la rappresentazione tabellare di Alistair Cockburn, che organizza la descrizione in campi predefiniti. Oltre lo scenario principale, è possibile mostrare quello delle estensioni,

ovvero i percorsi alternativi che si sviluppano da un determinato passo nel caso si verifichino determinate condizioni come, ad esempio, dati non validi, autorizzazioni insufficienti e altro ancora.

Sono stati selezionati due casi d'uso. Il primo, "Segnala issue", rappresenta la funzionalità principale del sistema ed è accessibile a tutti gli utenti autenticati che non hanno solo diritti di lettura. Il secondo, "Aggiorna lo stato", è stato scelto perché coinvolge il controllo degli accessi basato sui ruoli e include la notifica al segnalatore, che si attiva quando una segnalazione viene risolta.

2.5.1 Cockburn: Segnala una nuova issue

Use Case 1	Segnala issue		
Goal in Context	Inserimento nel sistema di una nuova segnalazione.		
Preconditions	L'utente è autenticato e non possiede il ruolo di sola lettura. La password temporanea è già stata modificata.		
Success End Conditions	La nuova segnalazione viene inserita con lo stato todo ; viene visualizzato correttamente l'utente segnalatore e la segnalazione è presente nell'elenco generale.		
Primary Actor	Utente		
Description	Step	Utente	Sistema
	1	Clicca "Nuova issue" su mock-up "Lista issue"	
	2		Mostra mock-up "Nuova Segnalazione" con il comando di conferma disattivato.
	3	Inserisce il titolo della segnalazione	
	4		Aggiorna il contatore dei caratteri residui.
	5	Inserisce la descrizione	
	6	Seleziona la tipologia	

	7		Evidenzia la tipologia e attiva il bottone di conferma
	8	Clicca "Crea segnalazione"	
	9		Verifica la validità dei dati inseriti
	10	Registra la segnalazione in stato <code>todo</code> associando l'utente che l'ha creata	
	11		Mostra mock-up "Dettaglio Issue" relativo alla segnalazione appena creata.
Extensions	Step	Utente	Sistema
Indicazione della priorità	6.1	Seleziona una priorità	
			Evidenzia priorità scelta
			Prosegue dal passo 8 registrando la segnalazione con la priorità indicata.
Titolo troppo breve	3.1	Inserisce titolo con meno di 3 caratteri	
			Mantiene disattivando il bottone di conferma
Titolo troppo lungo	3.2	Inserisce titolo con più di 200 caratteri	

			Impedisce l'inserimento e segnala il raggiungimento del limite massimo
Rinuncia all'inserimento	Ovunque	Clicca "Annulla" oppure "Torna alle segnalazioni"	
			Mostra mock-up "Lista issue"
Allegamento immagine	11.1	Seleziona un'immagine dall'area di caricamento	
			Ridimensiona e comprime immagine, poi la salva e modifica la issue
Immagine non valida	11.2	Seleziona un file diverso da un'immagine	
			Rifiuta l'immagine e segnala l'errore
Errore di comunicazione	9.1		Non riceve risposta dal server e segnala l'errore

Tabella 2.10: Tabella del caso d'uso "Segnala issue"

2.5.2 Cockburn: Assegna una issue e risoluzione

Use Case 2	Aggiorna stato		
Goal in Context	Modifica dello stato di una segnalazione a done da parte dell'assegnatario. Ne consegue l'invio di una notifica al segnalatore		
Preconditions	L'utente è autenticato e la segnalazione gli è stata assegnata da un amministratore.		
Success End Conditions	Lo stato della segnalazione viene aggiornato. Se viene risolta per la prima volta, viene memorizzato il momento in cui avviene e successivamente viene inviata una notifica al segnalatore.		
Primary Actor	Utente assegnatario		
Secondary Actor	Servizio di posta elettronica		
Description	Step	Utente Assegnatario	Sistema
	1	Clicca su una segnalazione da mock-up "Lista issue"	
	2		Verifica che l'utente sia un amministratore o l'assegnatario
	3		Mostra mock-up "Dettaglio Issue" con il pannello di modifica dello stato
	4	Seleziona il nuovo stato done dall'elenco	
	5		Verifica la validità dello stato inserito e i permessi dell'utente
	6		Registra il nuovo stato

	7		Registra il momento di risoluzione, se non è stata risolta in precedenza
	8		Genera una notifica destinata al segnalatore
	9		Trasmette al servizio di posta elettronica il messaggio da inviare
	10		Aggiorna mock-up "Dettaglio issue" con lo stato modificato
Extensions	Step	Utente Assegnatario	Sistema
Utente non ha i permessi	3.1		Omette il pannello di modifica dello stato
Utente seleziona uno stato diverso da done	6.1	Seleziona uno stato diverso da done	
			Registra il nuovo stato e prosegue con il passo 10, omettendo la generazione della notifica
Segnalazione già risolta in precedenza	7.1		Conserva il momento di risoluzione originario e prosegue con il passo 10, omettendo la generazione della notifica
Segnalatore e risolutore sono la stessa persona	8.1		Omette la generazione della notifica

Indisponibilità del servizio di posta	9.1		Registra l'anomalia e prosegue con il passo 10, omettendo la generazione della notifica
Segnalazione inesistente	2.1		Segnala l'errore
Modifica non autorizzata	5.1	Tenta la modifica senza averne i permessi	
			Rifiuta l'operazione e segnala l'errore

Tabella 2.11: Tabella del caso d'uso "Aggiorna lo stato" con risoluzione

2.6 Prototipazione visuale via Mock-up

La prototipazione dell'interfaccia grafica è stata realizzata in parallelo all'analisi dei requisiti. Lo strumento utilizzato per la realizzazione è Penpot, una piattaforma di design open-source gratuita. È stato deciso di non utilizzare Figma, per quanto le funzionalità e l'interfaccia siano alquanto identiche, in quanto il piano gratuito di Penpot offre maggiori servizi e non ha necessità di essere installato localmente.

Questo processo ha permesso di avere un quadro ben definito riguardo la concretizzazione dei requisiti identificati nel Capitolo 2.4, focalizzando l'attenzione sull'esperienza dell'utente in base alla frequenza d'uso prevista e al livello di competenza tecnica di ciascun profilo. Di seguito vengono riportati mock-up delle interfacce con le interazioni da esse.

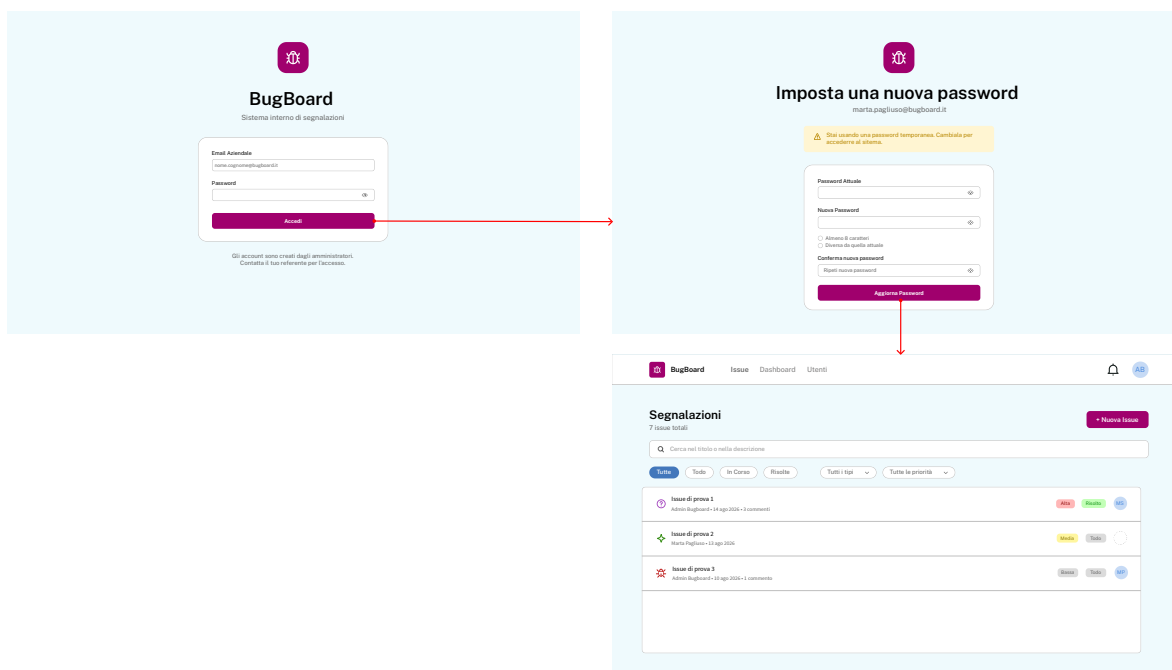


Figura 2.3: Mock-up relativi all'autenticazione

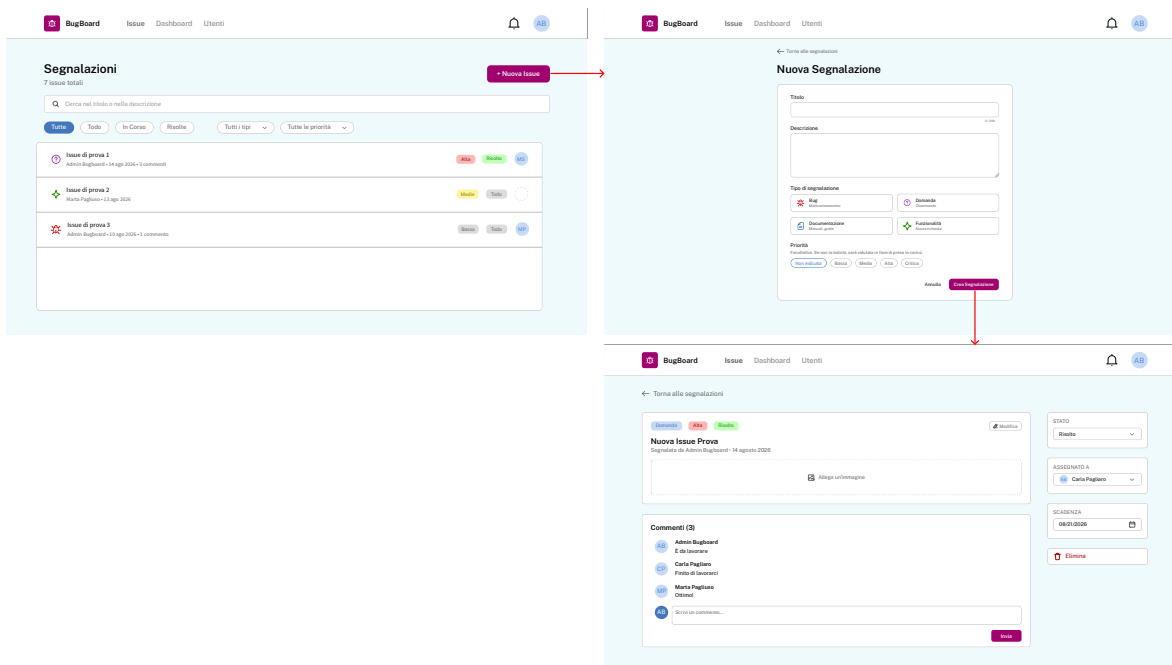


Figura 2.4: Mock-up relativi alle segnalazioni

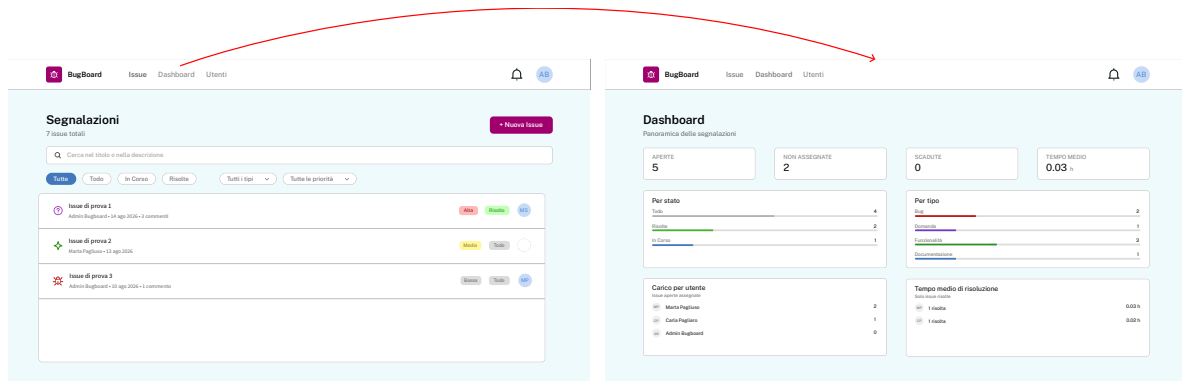


Figura 2.5: Mock-up relativi alla dashboard

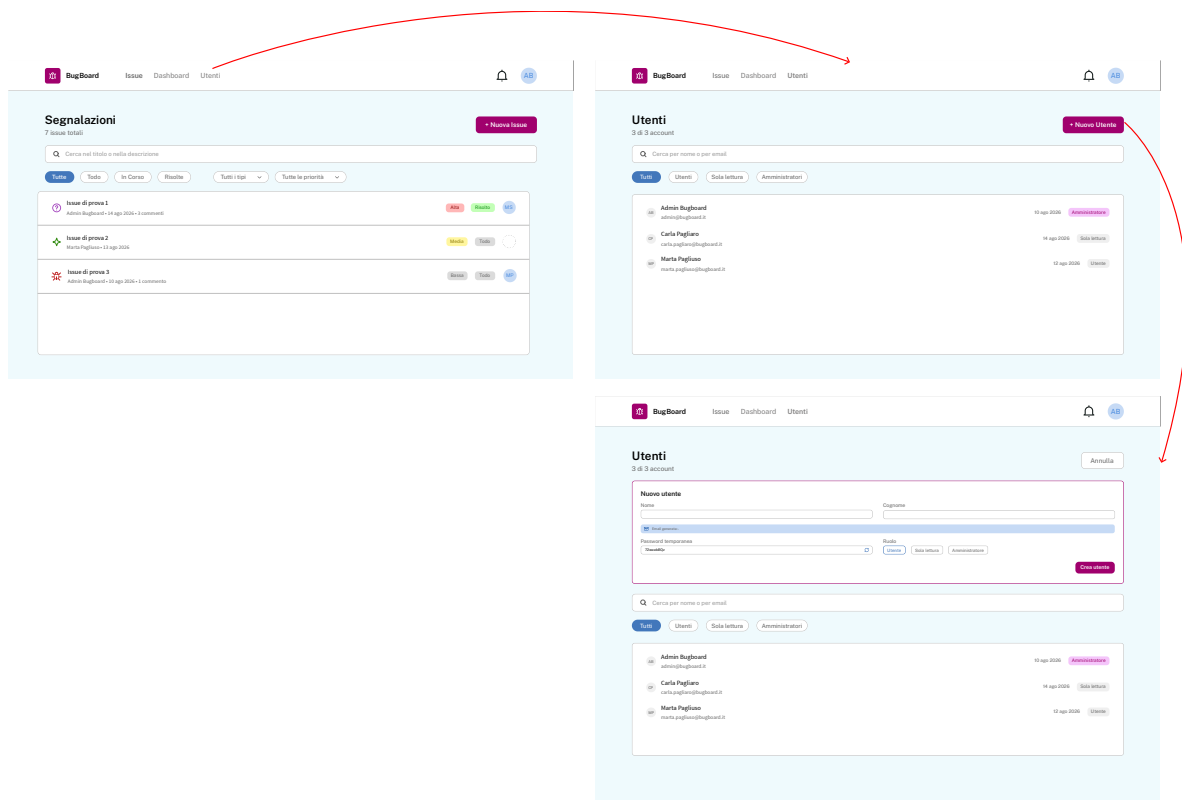


Figura 2.6: Mock-up relativi alla visualizzazione e creazione degli utenti

Capitolo 3

Documento di Design del Sistema

3.1 Progettazione architetturale e scelte di design

Il sistema presenta due componenti autonome: un back-end, che gestisce la logica business e la persistenza dei dati, e un front-end, che si occupa dell'interazione con l'utente. Queste comunicano tra di loro esclusivamente tramite rete, utilizzando un'interfaccia di programmazione che segue lo schema REST. Il front-end è completamente all'oscuro della struttura interna del back-end; allo stesso modo, quest'ultimo non conosce quale tipo di client utilizza i suoi servizi. Per aumentare la sicurezza nel sistema, è stato fatto in modo che il solo server abbia il compito di decidere quali operazioni sono ammissibili e di lasciare al client solo alcuni controlli più semplici.

I criteri che hanno guidato la progettazione di entrambe le componenti sono le seguenti:

- **Separazione delle responsabilità:** ogni unità ha un compito specifico e ben definito;
- **Disaccoppiamento:** un modulo sa quali risorse utilizza, ma non si occupa di chi lo utilizza;
- **Verificabilità:** il sistema è stato progettato in modo da essere testato automaticamente senza l'utilizzo di risorse esterne;
- **Estensibilità:** l'aggiunta di nuove entità porta il solo inserimento di nuovi moduli, evitando di modificare quelli esistenti.

3.1.1 Il Server

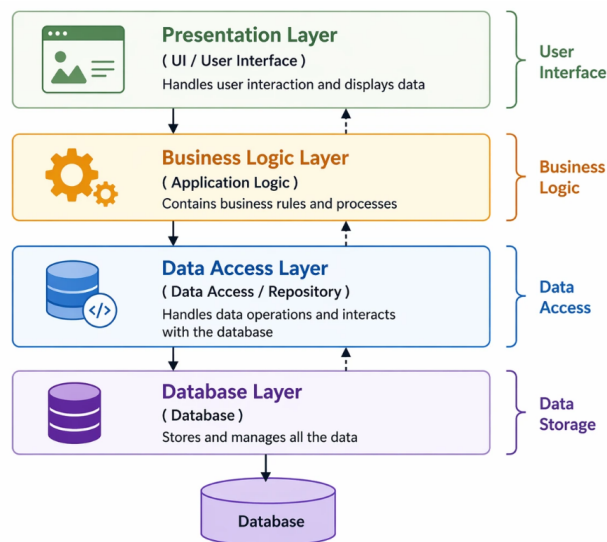
L'architettura implementata è quella a strati (*Layered Architecture*), in cui il sistema è diviso in livelli sovrapposti. Ciascuno di essi sfrutta le risorse date dai livelli inferiori e li passa a quelli superiori. Da questo si può dedurre che la comunicazione avviene solo tra livelli adiacenti.

Questo modello è stato scelto per due motivi. Il primo riguarda la natura del sistema, in quanto riceve richieste in formato uniforme e le gestisce attraverso un percorso lineare (validazione, applicazione delle regole di dominio, accesso ai dati). Il secondo riguarda la manutenibilità, in quanto si evita che una modifica a uno dei livelli possa ripercuotersi sugli altri.

Sono presenti quattro livelli, insieme a una serie di componenti trasversali:

1. **Presentation Layer (routes e controller)**: è il livello che si occupa della comunicazione con l'esterno. In particolare, le rotte intercettano le richieste HTTP in entrata e le indirizzano verso il gestore appropriato; i controller invece validano il contenuto della richiesta e invocano il livello sottostante traducendo il risultato nel codice di stato corretto;
2. **Application Layer (service)**: è il livello contenente la logica business. Qui avviene la trasformazione dei dati, il coordinamento tra operazioni diverse e si verifica se un'operazione è consentita.
3. **Data Access Layer (repository)**: è il livello che riceve i dati validati dai livelli precedenti e le restituisce alle strutture pronte all'uso. Sono presenti le operazioni CRUD con il database e utilizza un ORM (Drizzle) per astrarle.
4. **Database Layer**: è la base relazionale vera e propria.

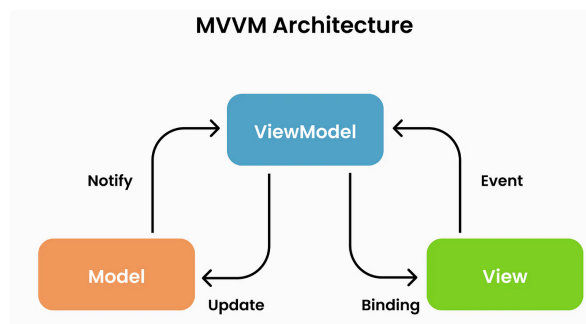
Le componenti trasversali sono i middleware. Si occupano di intercettare le richieste prima che arrivino al loro destinatario finale e svolgono alcuni compiti come la validazione dei token, controllare il ruolo dell'utente o bloccare gli account di sola lettura di effettuare modifiche.



3.1.2 Il Client

Poiché è previsto che il sistema venga utilizzato per molto tempo su poche schermate, il client è stato progettato come Single-Page Application, il che significa che l'app viene creata una sola volta e poi se ne aggiorna il contenuto in base alle richieste effettuate. Il vantaggio principale risiede nell'alleggerire il carico del server (data la diminuzione della quantità dei dati trasferiti), la reattività e la possibilità di utilizzare il sistema offline.

L'architettura interna segue il pattern MVVM (Model-View-ViewModel). Serve per separare la logica di presentazione (UI) dalla logica di business e dai dati dell'applicazione. Si compone, come è possibile vedere dal nome, di tre componenti: il Model rappresenta la struttura dati, le entità di dominio o le API utilizzate, e non conosce assolutamente l'interfaccia grafica; la View è l'interfaccia utente visiva; il ViewModel è l'intermediario tra il Model e la View. Trasforma i dati del Model in un formato consono per essere visualizzati dalla View e gestisce lo stato dell'interfaccia.



Sono presenti altri elementi trasversali, ma fondamentali per il funzionamento. I primi sono i service, che gestiscono la comunicazione con il back-end e di mantenere lo stato condiviso tra diversi componenti. Seguono il design pattern Singleton in quanto vengono creati una sola volta per l'intera applicazione. Un esempio è lo stato dell'utente autenticato, conservato in un unico service. Poi ci sono le guard, funzioni che il router consulta prima di attivare una rotta per controllare se l'accesso deve essere consentito o meno. Nel sistema ce ne sono due: il primo controlla se esiste una sessione valida, il secondo se l'utente è un amministratore. Infine c'è l'interceptor, che ha il compito di intercettare ogni richiesta HTTP in uscita e applicare una trasformazione uniforme, come l'aggiunta delle credenziali di sessione. Sono molto utili ai service, poiché evitano di dare loro la preoccupazione di gestire questa parte.

Per quanto riguarda le rotte, queste rispecchiano la struttura del sistema: le viste che precedono l'accesso si trovano al livello superiore e occupano l'intera finestra; le altre, invece, sono collegate a una rotta di layout, che si occupa di disegnare l'intestazione comune e di ospitare il contenuto variabile: in questo modo, l'intestazione viene definita una sola volta, evitando di ripeterla in ogni vista. È stato inoltre adottato l'approccio *lazy loading*, che consiste nel caricare la rotta solo su richiesta e non all'avvio dell'applicazione.

In conclusione, è stato scelto di memorizzare le credenziali in cookie che non possono essere accessibili dal codice del browser. Questa scelta nasce dalla necessità di aumentare la sicurezza in caso di robusti attacchi. Causa però una complicazione: il client non è in grado di capire se una sessione è attiva. Quando si ricarica la pagina, lo stato in memoria viene perso, mentre il cookie resta intatto. Per risolvere questo problema, si interroga il back-end all'avvio dell'applicazione, prima che il router viene attivato: se la richiesta per il profilo dell'utente va a buon fine, significa che la sessione è valida, altrimenti l'utente viene considerato come non autenticato.

3.2 Scelte tecnologiche adottate

La seguente sezione mira all'esposizione delle scelte tecnologiche adottate, con le relative motivazioni e alternative considerate.

3.2.1 Linguaggio e ambiente di esecuzione

Il linguaggio scelto è Typescript sia per la parte back-end sia per quella front-end. Si tratta di un'estensione di Javascript con un sistema di tipi statici e verificati in fase di

compilazione. Avendo usato per il back-end Express, si sarebbe potuto usare JavaScript per la suddetta parte e Typescript per l'altra. Non è stato preso in considerazione perché c'era un grande rischio di disallineamento tra le parti.

Per l'ambiente di esecuzione, è stato deciso Node.js per il suo modello di concorrenza non bloccante, particolarmente adatto a un'applicazione che si concentra principalmente sull'attesa di operazioni di input e output, come interrogazioni al database e invii di email, piuttosto che su elaborazioni intensive.

3.2.2 Framework di back-end

Come già espresso nel precedente paragrafo, Express è il framework impiegato per l'esposizione dell'interfaccia REST. I suoi vantaggi sono diversi: primo tra tutti la sua struttura minimale in quanto non impone un'architettura, lasciando che questa sia definita esplicitamente; secondo è il modello dei middleware che consente di realizzare la catena delle responsabilità.

Una valida alternativa sarebbe stata NestJS ma è stata preferita l'implementazione manuale dell'organizzazione a strati, il contenitore delle dipendenze e la gestione centralizzata degli errori che NestJS avrebbe offerto automaticamente.

3.2.3 Persistenza dei dati

La scelta su quale sistema di gestione della base di dati utilizzare è ricaduta su PostgreSQL. In particolare è stato utilizzato Neon come istanza ospitante. Questa scelta viene dalla volontà di accedere alla base di dati da qualsiasi ambiente di sviluppo senza la necessità di installazioni locali.

Altro strumento utilizzato è DrizzleORM che costituisce il livello di accesso ai dati. L'alternativa considerata è Sequelize, in quanto costituisce la soluzione storicamente più diffusa nell'ecosistema Node. È stato scartato per il suo supporto ai tipi aggiunto successivamente e non nativo. Questo avrebbe portato all'utilizzo di costrutti verbosi, mentre in Drizzle sono derivati dallo schema tramite `$inferSelect` e `$inferInsert`. Inoltre, nello specifico per la dashboard, servivano join con alias multipli sulla stessa tabella, `group by` su più colonne e sottoquery. Sequelize li supporta, ma con una sintassi che si allontana molto all'SQL prodotto, rendendo difficile prevedere e ottimizzare l'interrogazione.

3.2.4 Sicurezza e autenticazione

L'autenticazione avviene tramite JSON Web Token, seguendo il modello stateless. Il token è firmato con l'algoritmo HMAC-SHA256 e contiene l'identificativo dell'utente e il suo ruolo.

È stato deciso di utilizzare un modello a due token: il primo, chiamato "Access Token", ha una durata di quindici minuti e consente di effettuare operazioni su risorse protette; il secondo, denominato "Refresh Token", ha una durata di sette giorni e permette di ottenere un nuovo token senza dover ripetere l'autenticazione. Questa scelta viene dal fatto che i token, dopo essere stati emessi, non possono essere revocati fino alla scadenza. Se un token con una durata prolungata venisse rubato, potrebbe essere usato per l'intero periodo di validità. Per l'Access Token, il problema è contenuto, in quanto possiede una durata breve. Per il Refresh Token, invece, c'è un rischio molto alto. Il problema è stato risolto rendendolo una stringa casuale opaca, non un JWT, e il server ne conserva l'hash SHA256 nella riga dell'utente. Entrambi i token vengono trasmessi tramite cookie come `httpOnly` per renderli inaccessibili al codice JavaScript nel browser. L'alternativa del conservarli nel `LocalStorage` è stata scartata perché esporrebbe le credenziali a qualunque script malevolo eventualmente iniettato nella pagina. Inoltre, i cookie sono contrassegnati con l'attributo `sameSite=strict` che ne impedisce la trasmissione in richieste originate da siti terzi, fornendo protezione da attacchi di tipo cross-site request forgery.

Le password vengono salvate come impronte generate da `argon2id` con i seguenti parametri: 19Mib di memoria, due iterazioni e parallelismo unitario. A differenza delle funzioni di hash tradizionali, `argon2` è progettato per essere intenzionalmente pesante in termini di memoria e tempo, così da rendere impossibili gli attacchi che si basano su tentativi esaustivi.

3.2.5 Validazione dei dati

Per la validazione dei dati in ingresso è stato utilizzato `Zod`, una libreria che consente di dichiarare la struttura attesa di ogni richiesta e di verificarne l'adeguatezza prima che i dati raggiungano la logica applicativa. Un grande vantaggio risiede nel fatto che `Zod` genera automaticamente i tipi TypeScript dagli schemi di validazione garantendo sempre che la struttura dichiarata e i tipi siano sempre allineati. Un altro aspetto importante è la sicurezza, poiché scarta i campi non dichiarati. Dunque, se la richiesta include attributi non previsti, questi verranno semplicemente ignorati.

3.2.6 Gestione delle immagini

Per la gestione delle immagini, sono due gli strumenti adottati. Il primo è Multer, che si occupa della ricezione dei file. È impostato per mantenere i contenuti in memoria, invece di scriverli direttamente sul disco, permettendone l'elaborazione prima di salvarli. È stato inoltre impostata una grandezza massima di cinque megabyte. Il secondo è Sharp, che si occupa del ridimensionamento e compressione. Ogni immagine viene ridotta a un lato massimo di 1200 pixel e convertita in formato WebP, rendendola leggera ma ugualmente di qualità. Questo processo funge anche da validazione, dato che l'operazione fallisce se il file ricevuto non è in un formato corretto.

3.2.7 Notifiche per posta elettronica

Il servizio usato per le notifiche di posta elettronica è Nodemailer. In fase di sviluppo il traffico è indirizzato verso Mailpit, server SMTP fittizio che intercetta i messaggi e li rende consultabili mediante un'interfaccia web. Questa scelta deriva dal fatto che gli indirizzi email utilizzati sono fittizi, o meglio, appartengono a un dominio aziendale simulato.

3.2.8 Framework di front-end

Per la costruzione dell'interfaccia è stato scelto Angular, considerando come alternativa React. La decisione è stata presa valutando la struttura solida del framework, che offre nativamente un sistema di navigazione, un client HTTP e un meccanismo di iniezione delle dipendenze. Al contrario, React avrebbe richiesto l'integrazione di diverse librerie. Nello specifico sono stati utilizzati componenti standalone, che semplificano notevolmente la configurazione eliminando la necessità di dichiarare moduli. La gestione dello stato è affidata ai signal, i quali segnalano automaticamente i consumatori ogni volta che ci sono dei cambiamenti. Infine, insieme a RxJS, vengono gestite le operazioni asincrone che richiedono trasformazioni sul flusso, come la ricerca testuale.

Per quanto riguarda gli stili, è stato impiegato Tailwind CSS, definendo direttamente l'aspetto nel template e non in fogli di stile separati. La palette di colori del sistema è gestita tramite variabili, così da avere tutti i colori dichiarati in un unico punto.

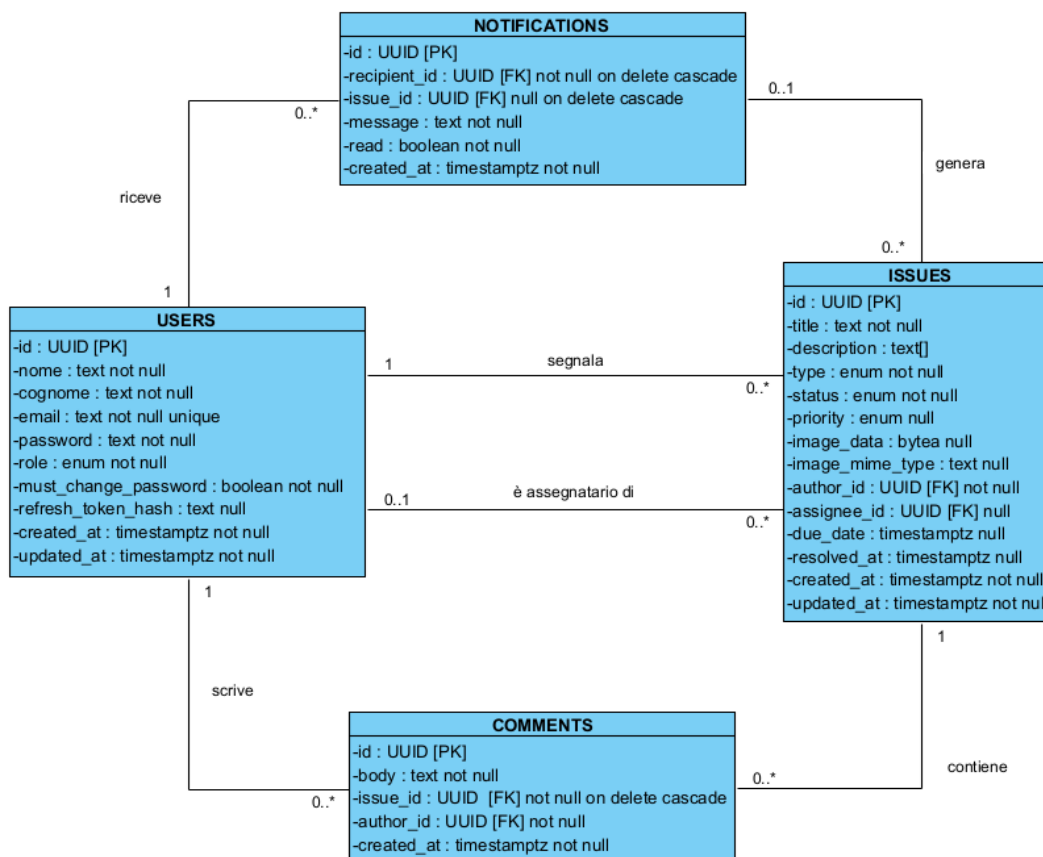
3.2.9 Strumenti di sviluppo e verifica

Per il controllo di versione è stato usato Git, con il suo repository ospitato su GitHub. L'organizzazione del lavoro si basa sul modello dei feature branch: ogni nuova funzio-

nalità viene sviluppata su un ramo dedicato e viene unita al ramo principale solo dopo aver superato i test. In merito ai test automatici, è stato scelto il framework Vitest. Nonostante Jest sia più conosciuto, è stato preferito Vitest per la sua compatibilità nativa con i moduli ECMAScript e TypeScript che ha permesso di evitare ulteriori configurazioni.

L'analisi statica del codice è stata affidata a SonarCloud, identificando potenziali difetti, vulnerabilità e aspetti che potrebbero compromettere la manutenibilità. Questa analisi è stata integrata tramite GitHub Actions che esegue la suite di test ogni volta che c'è una modifica nel ramo principale, calcola la copertura e invia i risultati alla piattaforma.

3.3 Diagramma per la persistenza dei dati



3.3.1 Descrizione del diagramma per la persistenza dei dati

Classe	Descrizione	Attributi
User	Rappresenta le persone abilitate all'accesso	id: UUID [PK] Identificatore univoco dell'utente nome: text [not null] Nome dell'utente cognome: text [not null] Cognome dell'utente email: text [not null, unique] Indirizzo di posta aziendale, generato automaticamente dal sistema password: text [not null] Password dell'utente prodotta dalla funzione argon2id role: enum [not null] Ruolo dell'utente. Può essere <code>user</code> , <code>admin</code> o <code>viewer</code> must_change_password: boolean [not null] Indica se l'utente deve modificare la password prima di operare refresh_token_hash: text [null] Impronta del token di rinnovo attualmente valido created_at: timestampz [not null] Momento della creazione dell'account updated_at: timestampz [not null] Momento dell'ultima modifica

Issue	Rappresenta una segnalazione inserita nel sistema	id: UUID [PK] Identificatore univoco di una segnalazione title: text [not null] Titolo sintetico della segnalazione (da 3 a 200 caratteri) description: text [not null] Descrizione estesa del problema type: enum [not null] Tipologia della segnalazione. Può essere <code>question</code>, <code>bug</code>, <code>documentation</code> o <code>feature</code> status: enum [not null] Stato di avanzamento. Può essere <code>todo</code>, <code>in_progress</code>, <code>done</code> o <code>closed</code> priority: enum [null] Grado di urgenza della segnalazione. Può essere <code>low</code>, <code>medium</code>, <code>high</code> o <code>critical</code> image_data: bytea [null] Contenuto binario dell'immagine allegata, già ridimensionata e compressa image_mime_type: text [null] Tipo di contenuto dell'immagine author_id: UUID [FK, not null] Riferimento all'utente che ha inserito la segnalazione assignee_id: UUID [FK, null] Riferimento all'utente a cui è stata affidata la segnalazione due_date: timestampz [null] Termine entro il quale la segnalazione deve essere risolta resolved_at: timestampz [null] Momento della prima risoluzione created_at: timestampz [not null] Momento di creazione della segnalazione updated_at: timestampz [not null] Momento dell'ultima modifica
--------------	---	---

Comment	Rappresenta un messaggio associato a una segnalazione	id: UUID [PK] Identificatore univoco del commento body: text [not null] Testo del commento issue_id: UUID [FK, not null, on delete cascade] Riferimento alla segnalazione a cui il commento fa riferimento author_id: UUID [FK, not null] Riferimento all'utente che ha creato il commento created_at: timestampz [not null] Momento di inserimento del commento
Notification	Rappresenta un avviso generato dal sistema e destinato a un determinato utente	id: UUID [PK] Identificatore univoco della notifica recipient_id: UUID [FK, not null, on delete cascade] Riferimento all'utente destinatario issue_id: UUID [FK, null, on delete cascade] Riferimento alla segnalazione che ha originato l'avviso message: text [not null] Testo dell'avviso read: boolean [not null] Indica se la notifica è stata letta created_at: timestampz [not null] Momento della generazione della notifica

3.4 Scelte di design dell'interfaccia utente adottate

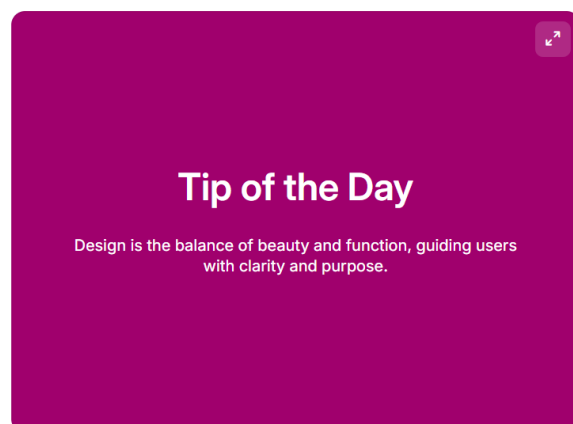
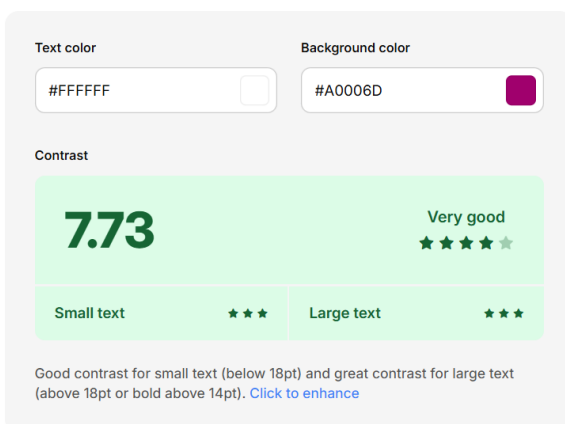
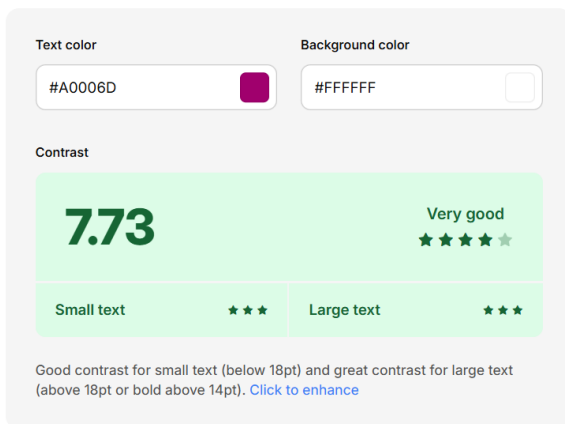
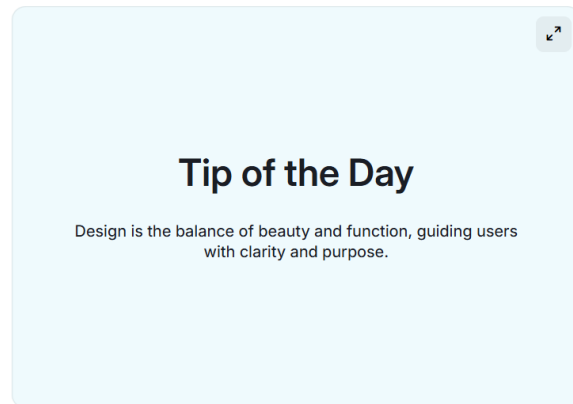
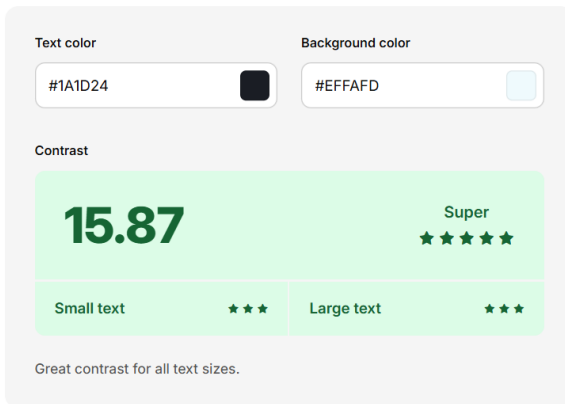
La progettazione dell'interfaccia utente è stata direzionata a partire dai profili descritti nella Sezione 2.3.1. È stata condotta prima della scrittura del codice, andando a descrivere la palette cromatica, la prototipazione delle schermate e la verifica dei requisiti di accessibilità. Di seguito sono esposte le scelte effettuate, ciascuna corredata dalla motivazione che l'ha determinata e, ove pertinente, dai vincoli che ne hanno condizionato la formulazione.

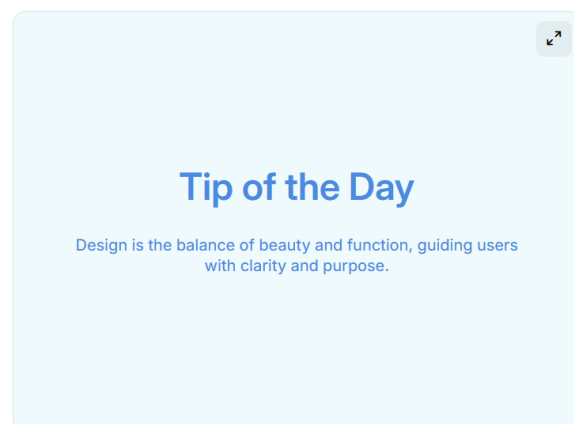
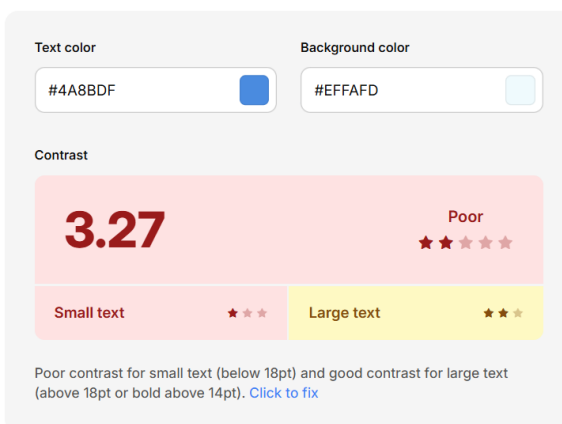
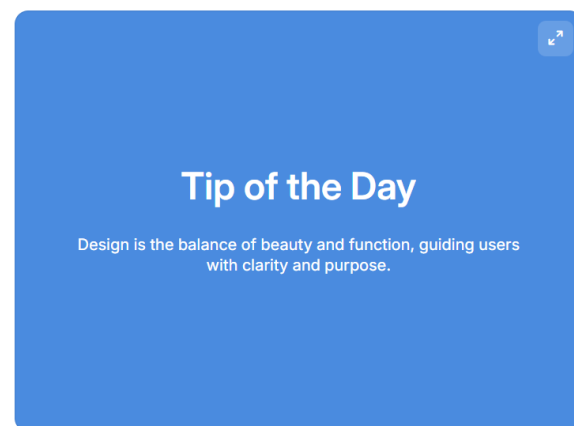
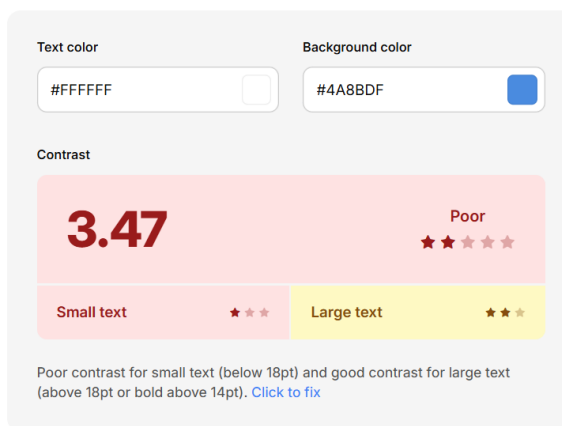
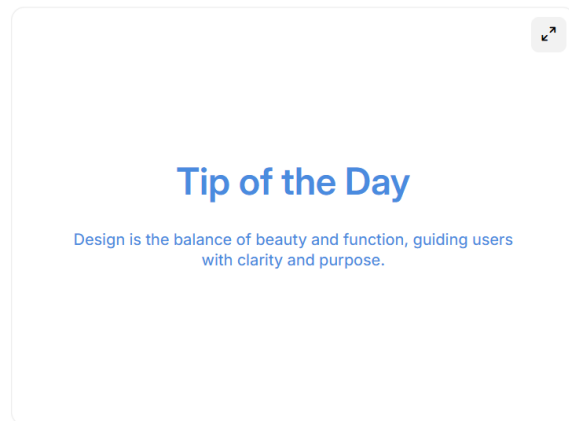
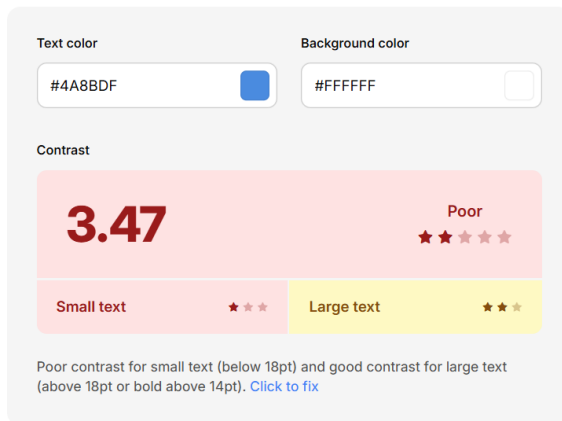
3.4.1 La palette cromatica

La scelta dei colori si è concentrata su tre tonalità: un azzurro delicato, un blu di media intensità e un magenta profondo. Questa combinazione cerca di trovare un equilibrio tra la freschezza dei primi due, che conferisce all'interfaccia un aspetto sobrio e professionale, e il calore del terzo, che aiuta a mettere in evidenza senza esitazioni l'elemento su cui vogliamo focalizzarci. Il loro uso segue la regola 60-30-10, un principio che afferma che il colore dominante dovrebbe occupare circa il sessanta percento dello spazio, uno secondario il trenta e un terziario il dieci percento. Nel design creato, l'azzurro funge da sfondo per le pagine, il blu per i bordi, elementi di navigazione e indicatori di stato, mentre il magenta è per le operazioni principali di ogni schermata.

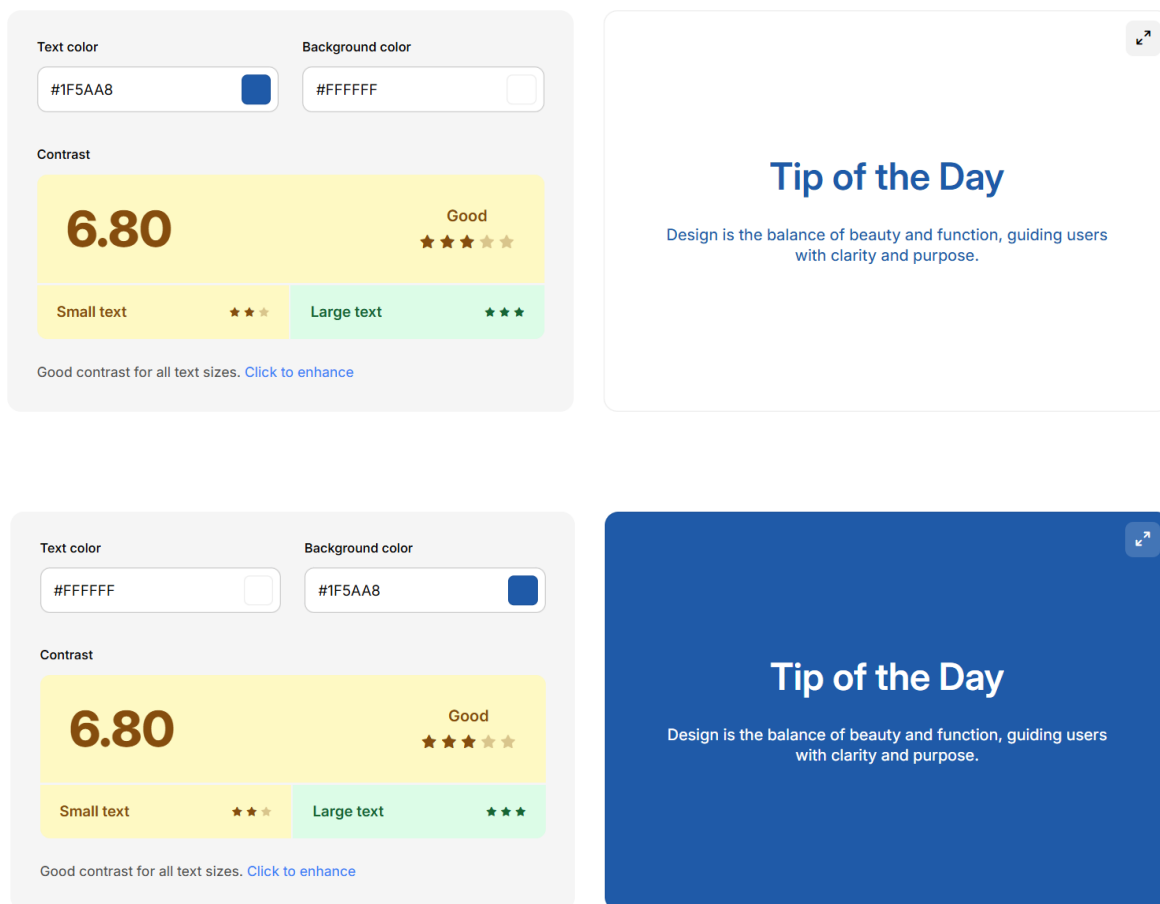
Ruolo	Tonalità	Impiego
Dominante (60%)	#EFFAFD	Sfondo delle pagine
Secondario (30%)	#4A8BDF	Bordi, indicatori di stato, elementi di navigazione
Accento (10%)	#A0006D	Azione primaria, notifiche non lette

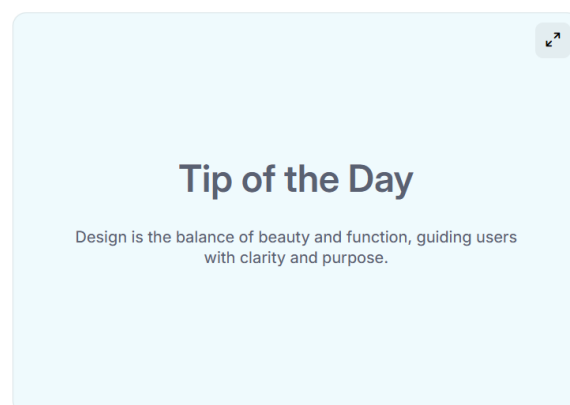
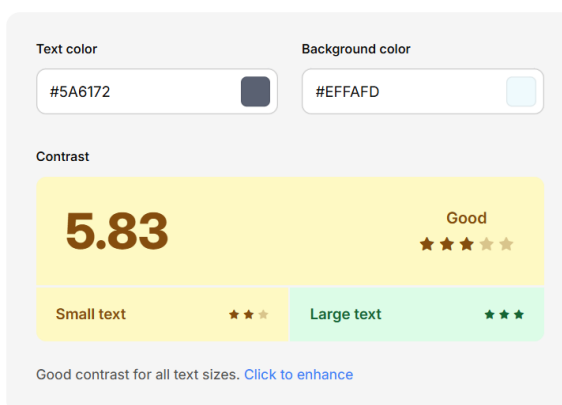
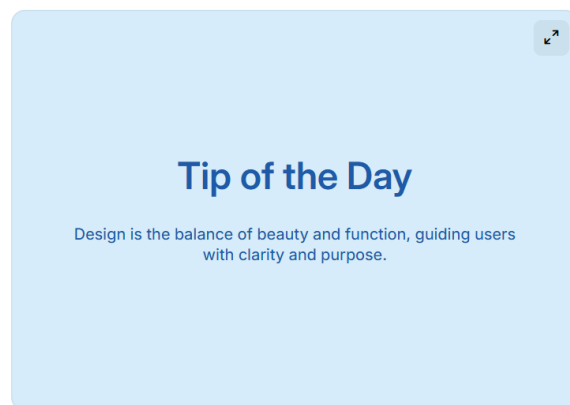
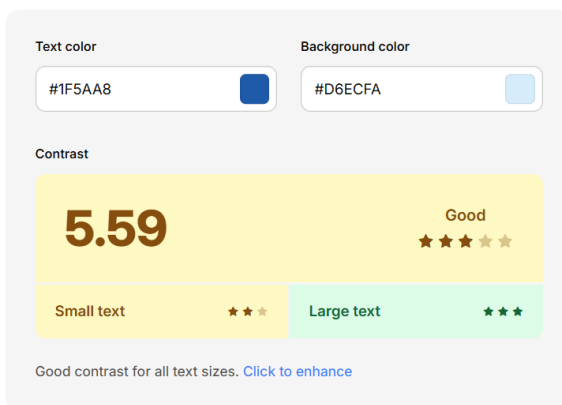
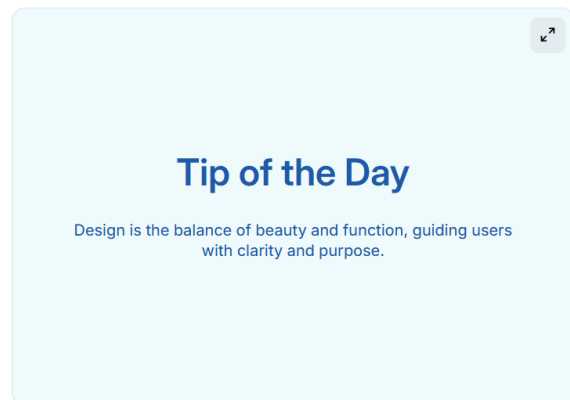
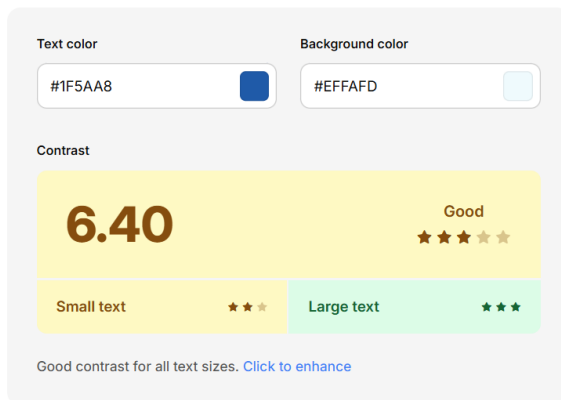
Prima di procedere all'implementazione, sono state controllate le combinazioni di tali colori con lo strumento Color Contrast Checker di Coolers, un tool online gratuito integrato nella piattaforma Coolers che verifica la leggibilità e l'accessibilità visiva di una combinazione di due colori (testo e sfondo). Calcola il rapporto di contrasto tra due tonalità e verifica se rispettano i criteri delle Web Content Accessibility Guidelines, i quali affermano che per raggiungere i livelli di conformità AA, il rapporto deve essere almeno di 4,5 : 1 per il testo di dimensioni normali e di 3 : 1 per il testo di grandi dimensioni per gli elementi non testuali. La verifica ha rivelato che la combinazione originaria non risultava conforme in tutti gli impieghi previsti.





Il magenta e il colore del testo si sono dimostrati perfettamente in linea; al contrario, il blu si è rivelato inadeguato in tutte le situazioni in cui doveva essere abbinato al testo. Non è una sorpresa poiché si tratta di una tonalità di luminosità intermedia, abbastanza chiara da non contrastare bene con il bianco e abbastanza scura per risaltare con uno sfondo azzurro. Ci sono state due possibili soluzioni. La prima prevedeva di sostituire completamente il blu con una tonalità più scura, ma questo avrebbe significato rinunciare alla vivacità che aveva reso questa scelta così attraente. La seconda, che abbiamo scelto, è stata quella di creare due varianti di blu destinate a usi diversi. Il blu originale è stato utilizzato per i bordi, sfondi leggeri e riempimenti, mentre per il testo e gli elementi interattivi è stato introdotto una variante con una luminosità ridotta, mantenendo però la stessa tonalità. Abbiamo quindi ripetuto la stessa verifica sulla palette così modificata.





La palette definitiva risulta pertanto costituita da sei tonalità, delle quali tre identitarie e tre derivate per esigenze di leggibilità.

Denominazione	Valore	Impiego
blue-50	#EFFFFD	Sfondo delle pagine
blue-100	#D6ECFA	Sfondo dei badge, elementi tenui
blue-400	#4A8BDF	Bordi, indicatori grafici, stato di focus
blue-700	#1F5AA8	Testo, collegamenti, elementi interattivi
plum-600	#A0006D	Azione primaria, notifiche non lette
plum-800	#6E004B	Stato di attivazione dell'azione primaria

3.4.2 Coerenza visiva tra le viste

Ogni elemento rimane sempre lo stesso nel sistema, rappresentato con gli stessi segni grafici, a prescindere dalla schermata in cui si trova. Ad esempio, lo stato di una segnalazione è rappresentato da un'etichetta il cui colore rimane costante tra l'elenco, il dettaglio e la dashboard; l'assegnatario è sempre rappresentato da un disco con le sue iniziali, e la tipologia è sempre indicata da un'icona fissa. Questa scelta aiuta a ridurre il carico cognitivo dell'utente e a rendere ogni elemento facilmente riconoscibile. Inoltre, la corrispondenza tra ogni stato e la sua rappresentazione grafica è definita in un'unica struttura all'interno della classe del componente, dalla quale il template attinge. L'altra opzione sarebbe stata ripetere questa corrispondenza direttamente nelle template, ma questo avrebbe portato una duplicazione sostanziosa della stessa informazione in ogni vista che mostra uno stato, portando una diminuzione della manutenibilità del codice.

3.4.3 Prevenzione dell'errore

Si è scelto, quando possibile, di bloccare un'azione piuttosto che permetterla e poi segnalare il risultato negativo. Questo principio, che nella letteratura si ricollega alla nozione di vincolo di Donald Norman, aiuta a ridurre la frustrazione dell'utente e il numero di tentativi andati a vuoto. Nel sistema, infatti, i comandi che avviano un'operazione sono disabilitati finché non vengono forniti i dati necessari: ad esempio, il pulsante per la creazione di una segnalazione si attiva solo quando titolo, descrizione e tipologia sono stati indicati. In questo modo, l'utente non deve tentare di inviare per scoprire che manca qualcosa.

Nel modulo per la sostituzione della password, invece, i requisiti richiesti vengono mostrati man mano che l'utente inserisce la nuova password, così da fornire un feedback immediato al posto di segnalare eventuali incongruenze solo alla fine e far ripetere l'operazione all'utente.

Nel modulo di creazione di un account, l'indirizzo email che il sistema genererà viene visualizzato e aggiornato mentre si digitano nome e cognome. Poiché l'indirizzo non è impostato dall'amministratore ma viene generato, l'anteprima rende chiaro un comportamento che altrimenti sarebbe poco trasparente.

Infine, il limite di lunghezza del titolo di una segnalazione è comunicato tramite un contatore che si aggiorna durante la digitazione, invece di un messaggio di errore che appare solo dopo l'invio.

3.4.4 Assenza in luogo della disabilitazione

Come già esplicitato nei paragrafi precedenti, non tutti gli utenti hanno gli stessi permessi. Un utente semplice, ad esempio, non può assegnare una segnalazione a un altro utente, non può visualizzare la dashboard e non può creare ulteriori utenti; un viewer, invece, non ha alcun permesso di modifica ma solo quello di visualizzazione. È stato deciso, infatti, di non solo disabilitare ma omettere completamente funzionalità che non sono permesse. Nel caso dell'utente, ad esempio, nella pagina di dettaglio di una segnalazione, non sono presenti i campi di selezione di un utente assegnatario o di inserimento di una data di scadenza.

Nel dettaglio, gli utenti in sola lettura non possono vedere il comando per inserire una nuova segnalazione né il modulo per scrivere commenti; la voce di navigazione per la vista di sintesi non appare a chi non ha un ruolo amministrativo, e il pannello per modificare lo stato è visibile solo a chi è responsabile della risoluzione. Tuttavia, all'utente in sola lettura viene mostrato un contrassegno che indica la sua condizione. L'assenza di comandi, se non spiegata, potrebbe risultare confusa; il contrassegno chiarisce in anticipo perché alcune operazioni non siano disponibili, evitando così che l'utente cerchi comandi che non esistono.

È importante notare che questo accorgimento riguarda solo la presentazione. L'autorizzazione effettiva viene verificata dal server per ogni richiesta, indipendentemente da ciò che l'interfaccia mostra: l'assenza di un comando è una cortesia verso l'utente, non una misura di sicurezza.

3.4.5 Riscontro sullo stato del sistema

L'interfaccia tiene sempre aggiornato l'utente sulla sua situazione, seguendo il primo dei principi euristici di Jakob Nielsen. Ogni operazione in corso mostra un messaggio che indica il suo andamento, ad esempio, si legge "Creazione..." e il comando viene disattivato evitando invii multipli.

Anche nelle situazioni in cui non ci sono risultati è presente un messaggio specifico invece di uno spazio vuoto. Nella lista delle segnalazioni, ad esempio, se un filtro non restituisce elementi, appare il messaggio "Nessuna issue trovata"; una segnalazione non assegnata è rappresentata da un disco tratteggiato, che indica l'assenza piuttosto che lasciare uno spazio vuoto che potrebbe sembrare un errore di caricamento.

È stata prestata particolare attenzione al feedback dopo la creazione di un account. Poiché l'indirizzo email è generato automaticamente e la password temporanea può essere creata dal sistema, l'amministratore potrebbe non avere le credenziali da comunicare al nuovo utente. Alla fine dell'operazione, viene quindi mostrato un messaggio che riporta entrambe le informazioni, rimanendo visibile fino a quando non viene chiuso esplicitamente.

3.4.6 Efficienza per l'uso ripetuto

Alcune scelte sono state effettuate per dare priorità alla velocità di esecuzione, anche se questo può sacrificare un po' la guida per l'utente, considerando quanto spesso ci si aspetta che venga utilizzato il sistema. L'elenco delle segnalazioni è ricco di informazioni: ogni riga include la tipologia, il titolo, l'autore, la data, il numero di commenti, la priorità, lo stato e l'assegnatario. I filtri sono facilmente utilizzabili come comandi, invece di essere nascosti in un pannello da aprire. La ricerca testuale avviene mentre si digita, senza bisogno di confermare esplicitamente, e c'è un piccolo ritardo che evita di inviare una richiesta per ogni singolo carattere. Inoltre, per assegnare una segnalazione a un utente, che richiederebbe di scegliere da un elenco potenzialmente lungo, è stato implementato un campo di ricerca in cui inserire alcune lettere del nome, cognome o email, senza dover scorrere l'intero elenco.

3.5 Diagramma delle classi di design

Il diagramma delle classi di design offre una visione panoramica della struttura del sistema, evidenziando le classi che lo compongono, le loro classi e le relazioni tra di esse. Data la grande quantità di classi, la loro rappresentazione è stata suddivisa in quattro diagrammi, ognuno dedicato a una funzione specifica. Le classi che interagiscono con gli altri diagrammi sono rappresentate in modo sintetico, poiché la loro struttura è descritta altrove. È importante specificare che non sono stati inclusi elementi privi di rilevanza architetturale: gli schemi di validazione, le strutture di trasferimento dati, costanti di presentazione utilizzati nei componenti nel client. Questa scelta è stata fatta per garantire maggiore leggibilità ai diagrammi che, altrimenti, sarebbero stati troppo corposi.

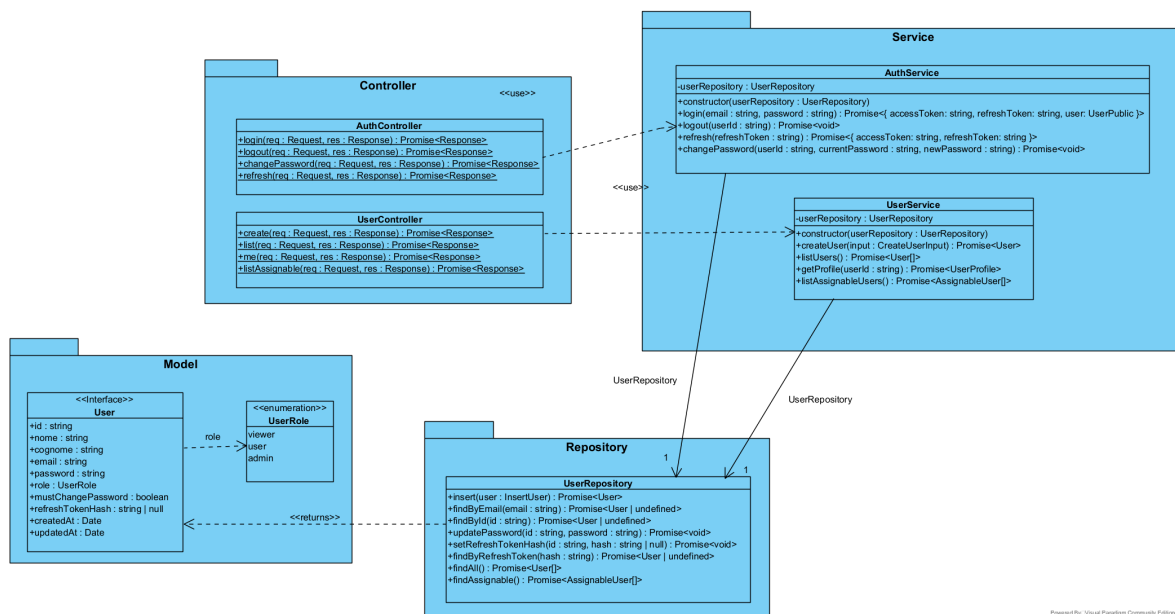


Figura 3.1: Diagramma riguardo l'autenticazione e gestione utenti

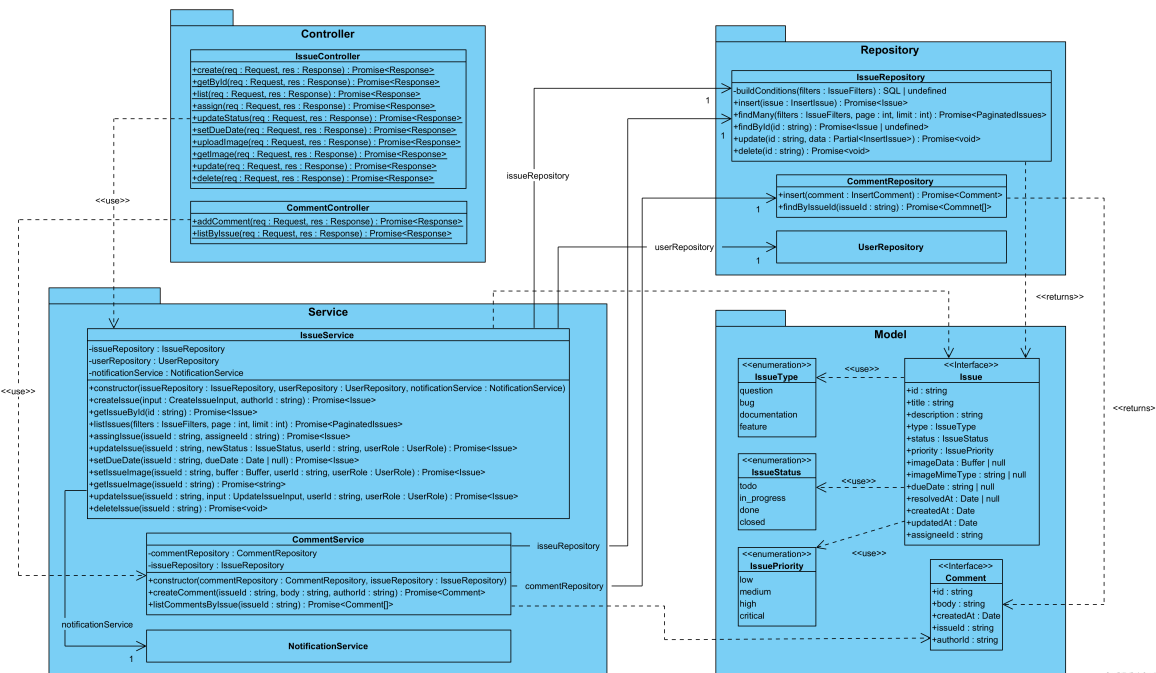


Figura 3.2: Diagramma riguardo le segnalazioni e i commenti

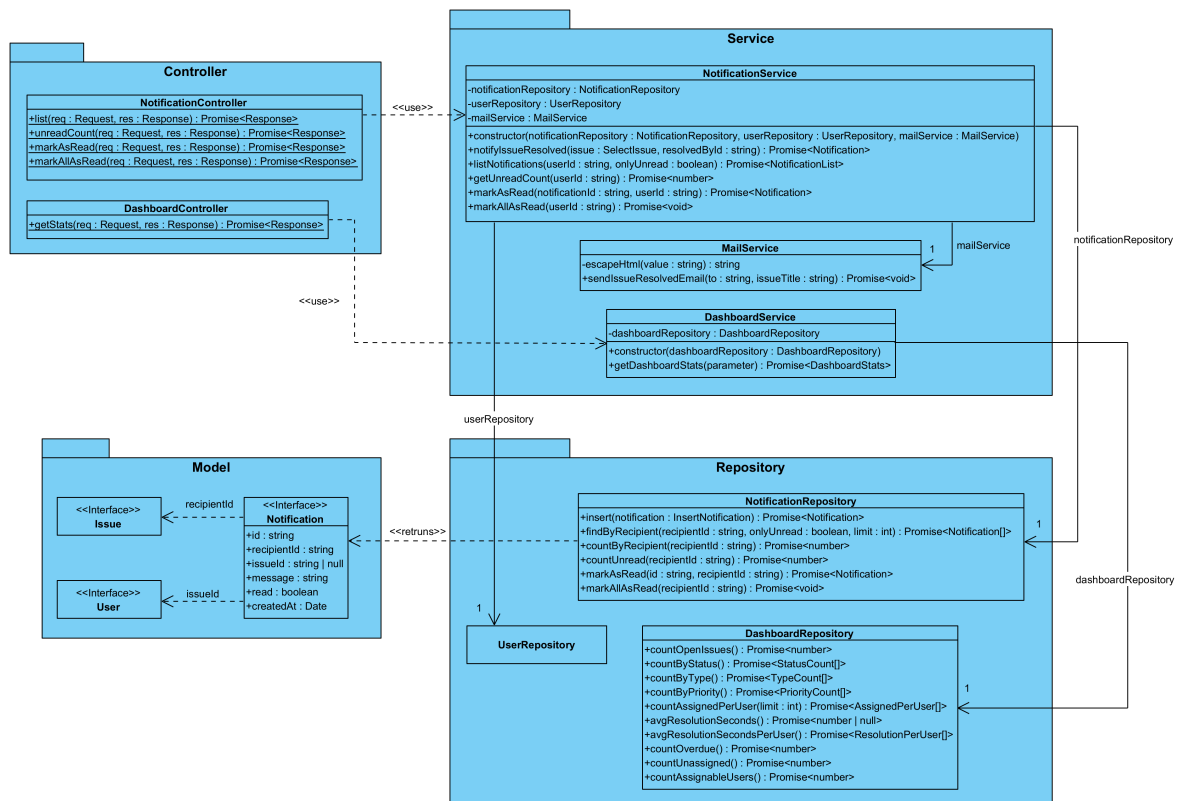


Figura 3.3: Diagramma riguardo le notifiche e monitoraggio

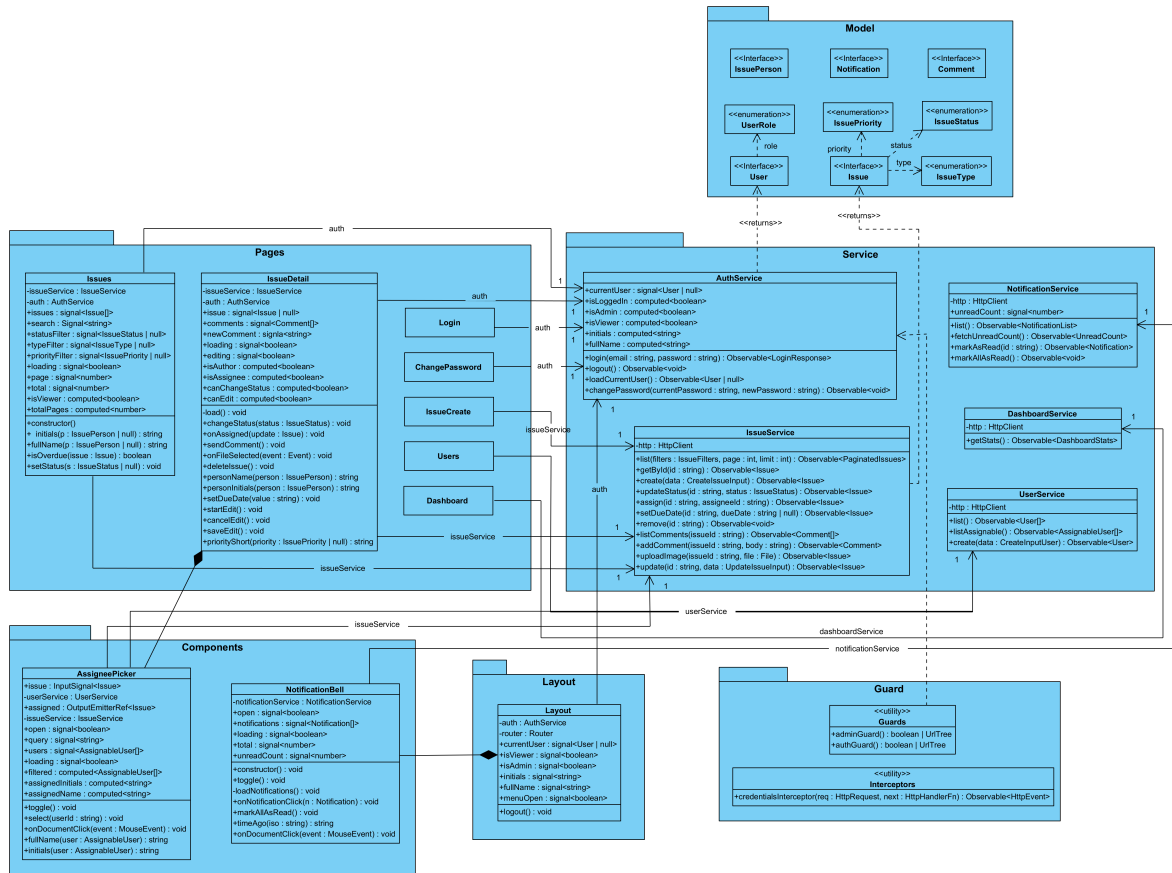


Figura 3.4: Diagramma riguardo il client

Capitolo 4

Processo di sviluppo

Il seguente capitolo si concentra su come è stata organizzata e verificata l'attività di sviluppo. Verrà trattata la gestione delle revisioni del codice, l'integrazione continua delle modifiche e il controllo automatico della qualità. Nonostante lo sviluppo sia stato portato avanti da una singola persona, è stato seguito un processo strutturato, sia per garantire una buona tracciabilità sia per poter isolare ogni modifica e risalire, in caso di problemi, all'origine.

4.1 Gestione del versioning

Per la gestione del versioning del codice è stato utilizzato Git, con un repository remoto ospitato su GitHub. È stato deciso di seguire il modello monorepo, il che significa che le due componenti del sistema (back-end e front-end) si trovano in cartelle separate all'interno dello stesso repository. Questa scelta non va in assoluto contrasto con la richiesta di indipendenza delle parti, in quanto si tratta di una autonomia riguardante l'esecuzione e la sostituibilità, non la posizione fisica del codice sorgente. Ogni cartella ha la sua configurazione delle dipendenze e un proprio processo di compilazione, permettendo di eseguire ciascuna parte in modo autonomo. Condividere il repository ha reso più semplice mantenere allineate le modifiche che riguardano entrambe le componenti.

È stato deciso di adottare il modello del feature branch, nel quale ogni funzionalità viene sviluppata su un ramo dedicato, creato a partire da un ramo principale. Solo quando la funzionalità è completa e testata, viene unita di nuovo al ramo principale. Così facendo, quest'ultimo mantiene sempre una versione funzionante del sistema. Durante lo sviluppo sono stati creati 25 rami, ognuno corrispondente a una specifica funzionalità.

Nella seguente tabella, vengono riportate le statistiche del repository.

Metrica	Valore
Commit complessivi	90
Rami creati	29
Contributori	1
Periodo di attività	Luglio-Agosto-Settembre 2026

Infine, è importante specificare che alcuni contenuti sono stati deliberatamente esclusi dal repository tramite il file `.gitignore`. Tra questi ci sono le dipendenze istallate, gli artefatti di compilazione, i rapporti di copertura generati ogni volta che si eseguono i test, le immagini caricate dagli utenti e il file con le variabili d'ambiente, che contiene le credenziali per l'accesso al database e il segreto utilizzato per firmare i token di sessione.

4.2 Report di qualità del codice

L'analisi statica del codice è un processo che prevede l'esame del sorgente senza eseguirlo, con l'obiettivo di scovare potenziali difetti, vulnerabilità e aspetti che potrebbero compromettere la manutenibilità. A questo scopo, è stato utilizzato SonarCloud, una versione del servizio offerta da SonarQube. L'analisi è integrata nel processo di sviluppo tramite GitHub Actions: ogni volta che c'è una modifica nel ramo principale, la pipeline si occupa di installare le dipendenze, eseguire la suite di test automatici, calcolare la copertura e inviare i risultati alla piattaforma. Seguendo le indicazioni del committente, l'analisi si concentra esclusivamente sul codice del back-end.

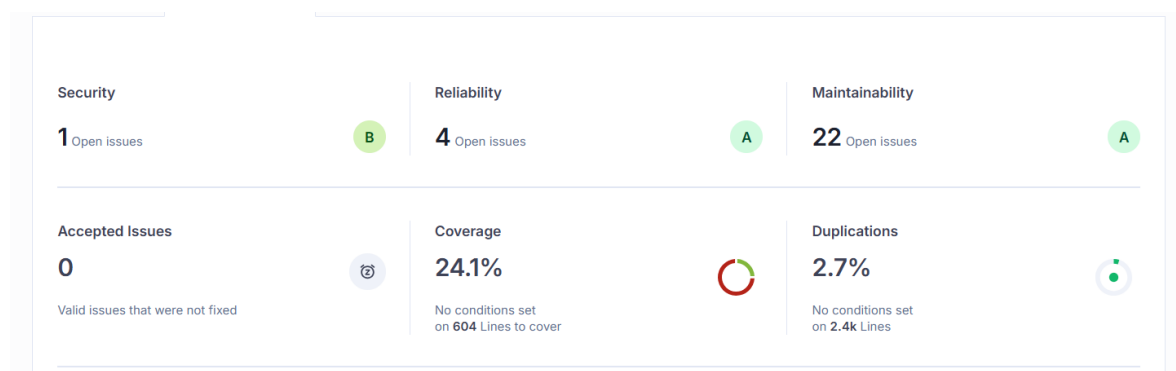
SonarCloud classifica le anomalie riscontrate in base a tre dimensioni di qualità, ognuna delle quali riceve un punteggio che va da A a E. La prima dimensione è Security, che si concentra sulle vulnerabilità, ovvero aspetti del codice che potrebbero compromettere il sistema; il secondo è Reliability, che riguarda i difetti che possono causare comportamenti anomali durante l'esecuzione; infine c'è Maintainability, che si occupa di quegli elementi che, pur non influenzando il funzionamento, rendono più difficile la comprensione e la modifica del codice. Ogni anomalia, inoltre, è accompagnata da un livello di severità e da una stima dello sforzo necessario per la correzione.

4.2.1 Stato rilevato dall'analisi iniziale

La prima analisi ha esaminato 1500 righe di codice e ha prodotto i seguenti risultati.

Dimensione	Valutazione	Anomalie rilevate
Security	B	1
Reliability	A	4
Maintainability	A	22

Metrica	Valore
Copertura delle verifiche automatiche	24,1%
Righe duplicate	2,7%



4.2.2 Anomalie rilevate e interventi adottati

Le anomalie si sono rivelate riconducibili a un numero limitato di regole, ciascuna violata in più punti del codice.

Regola	Dimensione	Severità	Volte	Esito
Divulgazione implicita delle informazioni di versione	Security	Minor	1	Corretta

Impiego di <code>String#replace()</code> in luogo di <code>String#replaceAll()</code>	Reliability, Maintainability	Minor	4	Corretta
Impiego improprio dell'operatore virgola	Maintainability	Major	1	Corretta
Eccezione intercettata e non gestita	Maintainability	Minor	2	Corretta
Catena di promise in luogo di attesa a livello di modulo	Maintainability	Major	1	Corretta
Importazione non utilizzata	Maintainability	Minor	2	Corretta
Impiego di API dichiarate deprecate	Maintainability	Minor	15	Accettata

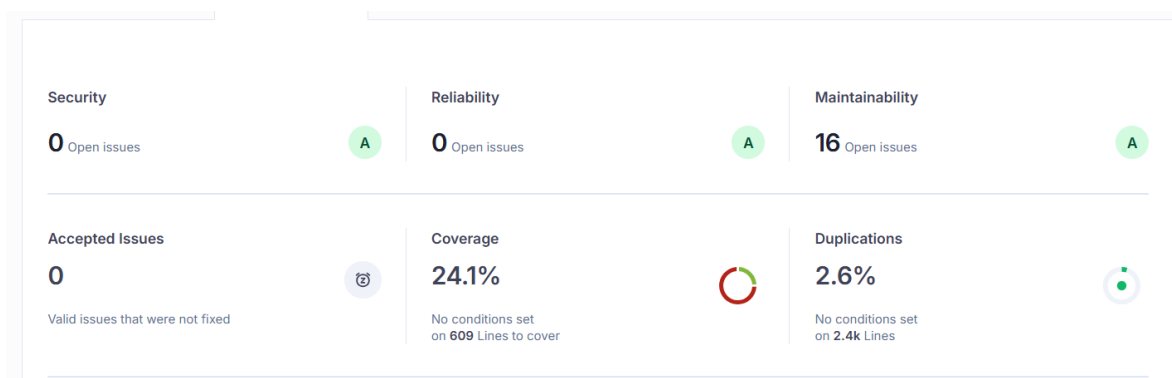
4.2.3 Stato conseguente agli interventi

Gli interventi correttivi sono stati raccolti in un ramo dedicato e sottoposti a nuova analisi mediante la medesima pipeline. Le modifiche hanno coperto, oltre le anomalie rilevate, anche i risultati dei test automatici descritti nella Sezione 5.1.2 e i test sull'usabilità descritti nella Sezione 5.2.

I risultati conseguiti sono riportati di seguito.

Dimensione	Valutazione iniziale	Anomalie iniziali	Valutazione finale	Anomalie finali
Security	B	1	A	0
Reliability	A	4	A	0
Maintainability	A	22	A	16

Metrica	Valore iniziale	Valore finale
Copertura delle verifiche automatiche	24,1%	24.1%
Righe duplicate	2,7%	2.6%



In conclusione, l'analisi statica del codice sorgente è stata utile per l'individuazione di un difetto funzionale che né la compilazione né i controlli automatici sono riusciti a scovare; e per la natura sistematica del controllo. Anomalie come importazioni non utilizzate o eccezioni intercettate ma non gestite, possono sembrare insignificanti da sole, ma la loro accumulazione nel tempo può compromettere la leggibilità del codice. Rilevarle automaticamente ad ogni modifica evita che si accumulino, senza necessitare di un'attività di revisione specifica.

Capitolo 5

Testing e valutazione dell'usabilità

5.1 Testing

L'attività di verifica è stata svolta su due fronti. Il primo prevede la creazione di un piano di test per una funzionalità completa, pensato per valutare come si comporta dal punto di vista dell'utente, includendo anche le situazioni di errore. Il secondo riguarda l'implementazione di test automatici su singoli metodi, che possono essere eseguiti più volte e integrati nel processo di sviluppo.

Nello specifico, il piano di test chiarisce cosa il sistema deve fare e in quali situazioni, fungendo da riferimento indipendente dall'implementazione; mentre i test automatici garantiscono che parti di codice funzionino come dovrebbero e che continuino a farlo dopo eventuali modifiche.

5.1.1 Test plan

La funzionalità scelta è l'aggiornamento dello stato di una segnalazione, che si ricollega al caso d'uso descritto nella Sezione 2.5.1. La scelta viene dal fatto che questa coinvolge molti aspetti contemporaneamente: il controllo degli accessi basato sui ruoli, la persistenza dei dati, la creazione di una notifica interna e l'invio di un'email.

L'obiettivo della verifica è quello di accertare che l'operazione venga consentita solo all'utente a cui è stata assegnata la segnalazione e agli amministratori, e negata a chiunque altro; che le informazioni vengano registrate correttamente, in particolare al momento della risoluzione; infine, che la generazione della notifica avvenga solo nei casi previsti, e un eventuale fallimento in questo processo non deve compromettere l'esito dell'operazione principale.

Per eseguire i casi di prova, è necessario un ambiente adeguatamente preparato. Servono almeno quattro utenze: un amministratore, due utenti ordinari e un utente in sola lettura. Questo perché alcune verifiche richiedono che il segnalatore e il risolutore siano persone diverse, mentre altre richiedono che siano la stessa persona. È inoltre necessario avere segnalazioni in stati diversi, di cui almeno una deve essere già stata dichiarata risolta in precedenza, per poter verificare il comportamento in caso di riapertura. Infine, il servizio di intercettazione dei messaggi di posta deve essere attivo, tranne nel caso di prova che verifica specificatamente la sua assenza.

I casi di prova individuati sono i seguenti.

ID	Descrizione	Precondizione	Azione	Esito atteso
TC01	Aggiornamento da parte dell'assegnatario	Segnalazione in stato todo assegnata all'utente	Selezione dello stato in lavorazione	Stato aggiornato, codice 200, momento della risoluzione non valorizzato
TC02	Prima risoluzione	Segnalazione in lavorazione assegnata all'utente	Selezione dello stato risolta	Stato aggiornato, momento della risoluzione valorizzato con l'istante corrente
TC03	Generazione della notifica interna	Come TC02, con segnalatore diverso dall'assegnatario	Come TC02	Notifica registrata e destinata al segnalatore
TC04	Recapito del messaggio di posta	Come TC03	Come TC02	Messaggio recapitato all'indirizzo del segnalatore, con titolo della segnalazione
TC05	Coincidenza tra segnalatore e risolutore	Segnalazione assegnata al proprio autore	Selezione dello stato risolta	Stato aggiornato, nessuna notifica generata

TC06	Aggiornamento da parte dell'amministratore	Segnalazione assegnata ad altro utente	Modifica dello stato da parte dell'amministratore	Operazione consentita, codice 200
TC07	Tentativo da parte di un utente non legittimata	Segnalazione assegnata ad altro utente	Tentativo di modifica da parte dell'utente ordinario	Operazione rifiutata, codice 403
TC08	Tentativo da parte di utente in sola lettura	Segnalazione qualsiasi	Tentativo di modifica	Operazione rifiutata, codice 403
TC09	Assenza di comando nell'interfaccia	Segnalazione non assegnata all'utente	Apertura del dettaglio	Il pannello di modifica dello stato non è presentato
TC10	Riapertura e successiva richiusura	Segnalazione già risolta	Ripristino dello stato <code>todo</code> e successiva nuova risoluzione	Il momento della risoluzione conserva il valore originario; nessuna nuova notifica
TC11	Segnalazione inesistente	-	Richiesta di aggiornamento su identificativo non presente	Operazione rifiutata, codice 404
TC12	Valore di stato non ammesso	Segnalazione assegnata all'utente	Trasmissione di un valore non previsto	Operazione rifiutata, codice 400

TC13	Indisponibilità del servizio di posta	Servizio di posta non in esecuzione	Dichiarazione di avvenuta risoluzione	Stato aggiornato e notifica interna registrata; anomalia annotata nel registro degli eventi
TC14	Richiesta priva di credenziali	-	Trasmissione della richiesta senza token di sessione	Operazione rifiutata, codice 401

Merita particolare attenzione il caso TC10, pensato per verificare una proprietà che non si evidenzia nel nell'uso quotidiano del sistema. Se una segnalazione già risolta viene aperta e poi chiusa di nuovo, il momento della risoluzione deve mantenere il valore originale registrato. Questo perché influisce sul calcolo del tempo medio di risoluzione mostrato nella dashboard per gli amministratori. Se il valore venisse sovrascritto, l'intervallo misurato inizierebbe dalla creazione della segnalazione fino all'ultima chiusura e non alla prima risoluzione. Si tratta di un difetto che non causerebbe malfunzionamenti evidenti, ma si manifesterebbe solo come un'inesattezza nelle statistiche.

Una considerazione simile si applica al caso TC13, che verifica una proprietà di resilienza. La consegna del messaggio di posta è un effetto secondario dell'operazione, non una sua condizione: quindi, il fallimento del servizio esterno non deve ostacolare né l'aggiornamento dello stato né la registrazione della notifica interna, che ha già raggiunto il suo obiettivo informativo.

Per quanto riguarda le modalità di esecuzione, i casi si suddividono in tre approcci diversi. I casi da TC01 a TC06, insieme al TC10, sono stati eseguiti manualmente tramite l'interfaccia utente, controllando sia lo stato del database e il contenuto della casella di posta intercettata. I casi TC07, TC08, TC11, TC12 e TC14 riguardano invece condizioni che l'interfaccia non permette di generare, poiché il comando non è accessibile agli utenti non autorizzati, e sono quindi stati eseguiti inviando direttamente le richieste all'interfaccia di programmazione. Il caso TC09 è stato verificato tramite ispezione visiva, mentre il TC13 ha richiesto un'interruzione deliberata del servizio di posta.

Infine, i casi TC01, TC02, TC05, TC06, TC07, TC10 e TC13, sono stati trasformati in verifiche automatiche, come descritto nella sezione successiva. Grazie a questa

automazione, possiamo rieseguire i test ogni volta che apportiamo modifiche al codice, mentre l'esecuzione manuale, a causa dei costi associati, tende a essere effettuata sempre meno frequentemente.

5.1.2 Strategie di progettazione dei test

La progettazione dei casi di prova è stata fatta seguendo criteri sistematici che hanno permesso di stabilire quali ingressi controllare e quando si potesse considerare sufficiente la verifica.

La prima strategia consiste nella suddivisione in classi di equivalenza, tipica della tecnica black-box. Qui i casi di prova vengono identificati a partire dalla specifica del metodo, senza tener conto dell'implementazione. Il dominio degli ingressi viene suddiviso in sottoinsiemi in cui ci si aspetta un comportamento uniforme. Si presume che, se il metodo funziona per un valore di una classe, allora funzionerà anche per gli altri valori della stessa classe. Di conseguenza, la verifica si concentra sulla selezione di un rappresentante per ciascuna classe.

La seconda è la copertura strutturale, tipica della tecnica white-box. In questo caso i casi di prova sono derivati dall'analisi del codice, in particolare dai percorsi che l'esecuzione può seguire attraverso le strutture di controllo. Il criterio utilizzato è quello della copertura delle decisioni, il che significa che ogni condizione presente nel metodo deve essere valutata almeno una volta come vera e almeno una volta come falsa.

Entrambe sono state applicate a diversi metodi. Per la funzione di generazione degli indirizzi di posta, che ha un ampio dominio di ingresso e una struttura di controllo semplice, è stata utilizzata la suddivisione in classi. Per i metodi dei servizi applicativi, che hanno ingressi più ristretti ma condizioni di autorizzazione complesse, è stata adottata la copertura strutturale.

La generazione degli indirizzi di posta

Il metodo `buildEmail` riceve nome e cognome di un utente, insieme a un eventuale numero progressivo per evitare ambiguità, e restituisce l'indirizzo email seguendo la struttura stabilita dal dominio aziendale. È importante verificarne il funzionamento, poiché normalizza i parametri ricevuti, che potrebbero includere caratteri non validi per un indirizzo email. Il dominio degli ingressi è stato ripartito nelle classi seguenti.

Classe	Rappresentante	Comportamento atteso
Nomi composti da sole lettere dell'alfabeto latino	Mario, Rossi	Trasformazione in minuscolo e concatenazione
Nomi contenenti caratteri accentati	Niccolò, Perù	Rimozione degli accenti
Nomi contenenti caratteri non alfabetici	Anna Maria, D'Amico	Rimozione di spazi e segni di interpunzione
Progressivo indicato	(Mario, Rossi, 2)	Apposizione del progressivo al cognome
Progressivo omesso	(Mario, Rossi)	Nessuna apposizione
Parametro vuoto	("", Rossi)	Sollevamento di eccezione
Caratteri estranei all'alfabeto latino	Caratteri cinesi	Sollevamento di eccezione

L'aggiornamento dello stato di una segnalazione

Il metodo `updateIssueStatus` presenta tre strutture condizionali, la cui combinazione determina cinque percorsi di esecuzione distinti.

Percorso	Condizione	Esito
P1	Segnalazione inesistente	Sollevamento di eccezione, nessuna scrittura
P2	Utente né assegnatario né amministratore	Sollevamento di eccezione, nessuna scrittura
P3	Transizione verso stato diverso dalla risoluzione	Aggiornamento senza registrazione del momento né notifica
P4	Prima transizione allo stato risolto	Registrazione del momento e generazione della notifica
P5	Segnalazione già risolta in precedenza	Conservazione del momento originario, nessuna nuova notifica

La sostituzione della password

Il metodo `changePassword` si basa su tre condizioni che vengono verificate in sequenza, e se una di esse è soddisfatta, l'esecuzione si interrompe: la mancanza dell'utente, l'erroneità della password attuale e la coincidenza tra la password attuale e quella nuova. A questi tre scenari negativi si aggiunge anche un percorso nominale, portando il totale a quattro casi.

In aggiunta è stata implementata una verifica di tipo diverso, che si concentra sulla sicurezza, ovvero l'assicurarsi che il valore inviato allo strato di persistenza sia l'impronta crittografica della password e non la password in chiaro. Questa verifica non è legata a un criterio di copertura, ma si basa sull'identificazione di un'invariante che il sistema deve mantenere.

Isolamento dalle dipendenze esterne

I metodi dei servizi applicativi si basano sui rispetti repository per accedere al database. Poiché una verifica che richiedesse la disponibilità del database sarebbe stata lenta, è stato deciso di sostituire queste dipendenze con implementazioni fittizie. Questa scelta è stata possibile grazie alla struttura architetturale descritta del capitolo precedente, in quanto i servizi non creano i propri repository, ma li ricevono tramite il costruttore. Durante le verifiche, è quindi sufficiente fornire un oggetto che esponga gli stessi metodi restituendo valori predefiniti. Questa sostituzione permette anche di controllare non solo il valore restituito del metodo, ma anche le interazione con le sue dipendenze: quali metodi sono stati chiamati, con quali argomenti e quante volte. In questo modo, ad esempio, possiamo assicurarci che la notifica non venga generata se il segnalatore è lo stesso del risolutore. Infine, la riproducibilità è garantita dalla configurazione delle variabili d'ambiente in un modulo di inizializzazione dedicato, con valori fissi che non dipendono dalla configurazione della macchina su cui viene eseguita la suite.

5.1.3 Test di unità automatici

I test di unità automatici sono stati effettuati utilizzando Vitest, un framework che fa parte della famiglia xUnit. La suite di test si trova in una cartella separata rispetto ai sorgenti, seguendo una struttura organizzativa ben definita, e comprende quarantotto casi distribuiti su cinque file. Di seguito, troverete gli estratti più significativi, accompagnati dalle riflessioni che hanno guidato la loro scrittura. Il codice completo è disponibile nel repository.

Verifiche derivate dalle classi di equivalenza

Ogni classe identificata nella sezione precedente viene esaminata attraverso un caso specifico. La corrispondenza è chiaramente indicata tramite commenti, in modo che sia facile verificare a quale classe appartiene ciascun caso.

```
1 describe('buildEmail', () => {
2   // Classe di equivalenza = nomi validi, solo lettere latine
3   it('Genera l\'email nel formato nome.cognome@bugboard.it', () => {
4     expect(buildEmail('Mario', 'Rossi')).toBe('mario.rossi@bugboard.it');
5   });
6
7   // Classe di equivalenza = caratteri accentati, da normalizzare
8   it('Rimuove gli accenti dai caratteri', () => {
9     expect(buildEmail('Niccolò', 'Perù')).toBe('niccolo.peru@bugboard.it');
10  });
11
12  // Classe di equivalenza = caratteri non alfabetici (apostrofi, spazi)
13  it('Rimuove apostrofi e spazi', () => {
14    expect(buildEmail('Anna Maria', "D'Amico")).toBe('annamaria.damico@bugboard.it');
15  });
16
17  // Classe di equivalenza = progressivo per omonimia
18  it('Aggiunge un progressivo quando ci sono nomi uguali', () => {
19    expect(buildEmail('Mario', 'Rossi', 2)).toBe('mario.rossi2@bugboard.it');
20  });
21
22  // Classe di equivalenza = progressivo non inserito
23  it('Non aggiunge nulla se il suffisso è undefined', () => {
24    expect(buildEmail('Mario', 'Rossi', undefined)).toBe('mario.rossi@bugboard.it');
25  });
26
27  // Classe di equivalenza = nome vuoto (invalido)
28  it("Lancia '[!]' Nome non valido' con nome vuoto", () => {
29    expect(() => buildEmail('', 'Rossi')).toThrow('[!] Nome non valido');
30  });
31
32  // Classe di equivalenza = cognome vuoto (invalido)
33  it("Lancia '[!]' Nome non valido' con cognome vuoto", () => {
34    expect(() => buildEmail('Mario', '')).toThrow('[!] Nome non valido');
35  });
36
37  // Classe di equivalenza = solo caratteri non latini (invalido)
38  it("Lancia '[!]' Nome non valido' con caratteri non latini vuoto", () => {
39    expect(() => buildEmail('王', '李')).toThrow('[!] Nome non valido');
40  });
41 });
```

Sostituzione delle dipendenze

I metodi dei servizi applicativi necessitano di sostituire i repository con delle implementazioni fittizie. Queste vengono ricostruite prima di ogni caso, in modo che lo stato generato da una verifica non influisca su quelle successive.

```
1 describe('IssueService.updateIssueStatus', () => {
2   let issueRepository: any;
3   let userRepository: any;
4   let notificationService: any;
5   let service: IssueService;
6
7   // Serve per ricreare i mock prima di ogni test
8   beforeEach(() => {
9     issueRepository = {
10       findById: vi.fn().mockResolvedValue({ ...baseIssue }),
11       update: vi.fn().mockResolvedValue(undefined),
12     };
13
14     userRepository = { findById: vi.fn() };
15     notificationService = { notifyIssueResolved: vi.fn().mockResolvedValue(undefined) };
16     service = new IssueService(issueRepository, userRepository, notificationService);
17   });
```

Verifiche di percorso

Ciascun percorso individuato nell'analisi strutturale è verificato da un caso distinto. Si riportano i due più significativi.

```
1 // Percorso 2: l'utente non è né un assegnatario né un admin => autorizzazione negata
2 it("Lancia '[!] Vietato' se l'utente non è assegnatario né admin", async () => {
3   await expect(
4     service.updateIssueStatus('issue-1', 'done', 'estraneo-1', 'user'),
5     ).rejects.toThrow('[!] Vietato');
6
7   expect(issueRepository.update).not.toHaveBeenCalled();
8 });
```

L'asserzione finale non si concentra sul valore restituito, ma sull'assenza di un'interazione. Verifica infatti che il tentativo non autorizzato non abbia portato a nessuna scrittura. Se ci si limitasse a controllare solo il sollevamento dell'eccezione, non si potrebbe escludere che la modifica sia avvenuta prima del rifiuto.

```
1 // Percorso 4: la issue è già stata risolta una volta => resolvedAt invariato e nessuna notifica
2 it("Non sovrascrive resolvedAt né rimanda la notifica se la issue è già risolta", async () => {
3   const risolta = new Date('2026-08-05T09:00:00Z');
4   issueRepository.findById.mockResolvedValue({ ...baseIssue, status: 'done', resolvedAt: risolta });
5
6   await service.updateIssueStatus('issue-1', 'done', 'assignee-1', 'user');
7
8   const [, payload] = issueRepository.update.mock.calls[0];
9   expect(payload.resolvedAt).toBe(risolta);
10
11   expect(notificationService.notifyIssueResolved).not.toHaveBeenCalled();
12 });
```

Verifica del comportamento in caso di fallimento

```
1 it("Completa il cambio di stato anche se la notifica fallisce", async () => {
2   notificationService.notifyIssueResolved.mockRejectedValue(new Error('SMTP down'));
3
4   await expect(
5     service.updateIssueStatus('issue-1', 'done', 'assignee-1', 'user')
6   ).resolves.not.toThrow();
7
8   expect(issueRepository.update).toHaveBeenCalled();
9 });
```

Il caso dimostra una caratteristica di resilienza: la controfigura è progettata per fallire, ma sorprendentemente, l'operazione principale riesce comunque a concludersi con successo. Questo è in linea con il caso di prova TC13.

Verifica di proprietà di sicurezza

Le verifiche non si basano su criteri di copertura, ma piuttosto sull'identificazione di invarianti che il sistema deve mantenere.

```
1 it('Salva un hash argon2, non la password in chiaro', async () => {
2   await service.changePassword('user-1', 'passwordAttuale', 'passwordNuova');
3
4   const [id, hashSalvato] = userRepository.updatePassword.mock.calls[0];
5   expect(id).toBe('user-1');
6   expect(hashSalvato).not.toBe('passwordNuova');
7   expect(hashSalvato).toMatch(/^$argon2id$/);
8 });
```

```
1  it("Non espone password e refresh token", async () => {
2      userRepository.findById.mockResolvedValue({
3          id: 'u1', nome: 'Mario', cognome: 'Rossi',
4          email: 'mario.rossi@bugboard.it', role: 'user',
5          password: '$argon2id$hash', refreshTokenHash: 'abc123',
6          mustChangePassword: false, createdAt: new Date(),
7      });
8
9      const profile = await service.getProfile('u1');
10
11      expect(profile).not.toHaveProperty('password');
12      expect(profile).not.toHaveProperty('refreshTokenHash');
13      expect(profile.email).toBe('mario.rossi@bugboard.it');
14  });
```

Il primo punto assicura che la password non venga mai inviata in caso allo strato di persistenza, mentre il secondo garantisce che il profilo restituito al client non contenga informazioni riservate.

Verifica di comportamenti iterativi

```
1  it("Aggiunge il progressivo quando l'email è già occupata", async () => {
2      userRepository.insert
3          .mockRejectedValueOnce(uniqueViolation())
4          .mockResolvedValueOnce({ id: 'u2', email: 'mario.rossi2@bugboard.it' });
5
6
7      await service.createUser({ nome: 'Mario', cognome: 'Rossi', password: 'password123' });
8
9      expect(userRepository.insert).toHaveBeenCalledTimes(2);
10     expect(userRepository.insert.mock.calls[0][0].email).toBe('mario.rossi@bugboard.it');
11     expect(userRepository.insert.mock.calls[1][0].email).toBe('mario.rossi2@bugboard.it');
12  });
```

La controfigura è impostata in modo che il primo tentativo di inserimento fallisca, simulando una violazione del vincolo di unicità, mentre il secondo vada a buon fine. In questo modo, quando si attiva il meccanismo di disambiguazione degli omonimi, che normalmente nel sistema si attiverebbe raramente e la cui correttezza sarebbe altrimenti complicata da verificare.

Riproducibilità

L'esecuzione della suite deve essere indipendente dalla configurazione della macchina e dalla disponibilità di risorse esterne. Per questo motivo, le variabili d'ambiente necessarie ai moduli sono impostate con valori fissi in un file di inizializzazione, che viene caricato prima di ogni file di verifica.

```
1 // tests/setup.ts
2 process.env.JWT_SECRET = 'test-secret-lungo-almeno-32-caratteri-per-sicurezza';
3 process.env.SMTP_HOST = 'localhost';
4 process.env.MAIL_FROM = 'test@bugboard.it';
5 process.env.DATABASE_URL = 'postgresql://test:test@localhost:5432/test';
```

I valori sono puramente fittizi e non viene stabilita alcuna connessione reale: servono solo a garantire che le verifiche di configurazione effettuate dai moduli al momento del caricamento possano andare a buon fine.

5.1.4 Risultati e difetti individuati

La suite di verifica si compone di quarantotto casi, distribuiti secondo la tabella seguente.

Unità di verifica	Casi	Strategia adottata
buildEmail	8	Suddivisione in classi di equivalenza
IssueService	20	Copertura strutturale
UserService	10	Copertura strutturale e verifica di invarianti
AuthService	5	Copertura strutturale e verifica di invarianti
CommentService	5	Copertura strutturale
Totale	48	

Di seguito, invece, viene riportata la copertura ottenuta sui moduli che sono stati verificati. È importante notare che questa copertura non si applica all'intero sistema, poiché l'attività è circoscritta ai metodi specificati dal committente e ad alcuni altri che sono stati considerati significativi.

Modulo	Line Coverage	Function Coverage	Branch Coverage
auth.service.ts	37.0%	40.0%	50.0%
comment.service.ts	100%	100%	100%
dashboard.service.ts	83.3%	75.0%	50.0%
issue.service.ts	63.5%	46.2%	62.3%
mail.service.ts	0.0%	0.0%	100%
notification.service.ts	0.0%	0.0%	0.0%
user.service.ts	85.0%	66.7%	92.9%
email.ts	100%	100%	100%
jwt.ts	40.0%	0.0%	50.0%
mailer.ts	0.0%	100%	100%
password.ts	100%	100%	100%
refresh.ts	50.0%	0.0%	100%

I moduli che presentano una copertura nulla o parziale sono quelli per cui la verifica avrebbe richiesto risorse esterne, come ad esempio il servizio postale. Oppure, si tratta di moduli composti esclusivamente da invocazioni allo strato di persistenza, dove una verifica isolata confermerebbe solo la corretta delega, senza coinvolgere alcuna logica applicativa.

L'attività di verifica ha portato all'individuazione di due anomalie, una riguardante un difetto funzionale, l'altra una carenza di verifica.

Il controllo di validità sui parametri non normalizzati

Il caso di prova relativo alla classe di equivalenza dei caratteri non appartenenti all'alfabeto latino ha portato a un risultato diverso da quello che ci si aspettava: invece di sollevare l'eccezione prevista, il metodo restituiva un indirizzo malformato. Analizzando il codice, è emerso che il controllo di validità si applicava ai parametri ricevuti, piuttosto che ai valori ottenuti dalla normalizzazione.

```
1 const nomeFinale = slugify(nome);
2 const cognomeFinale = slugify(cognome);
3
4 if (!nome || !cognome)           // difetto: verifica i parametri originari
5   throw new Error('[!] Nome non valido');
```

Poiché i parametri contenevano caratteri — anche se non facevano parte dell'alfabeto latino — il controllo sembrava andare a buon fine, mentre i valori normalizzati risultavano essere stringhe vuote. Di conseguenza, il metodo restituiva l'indirizzo `@bugboard.it`, che, senza ulteriori verifiche, sarebbe stato registrato nel database come identificativo di accesso di un utente. La correttezza ha comportato la verifica dei valori normalizzati:

```
1 if (!nomeFinale || !cognomeFinale)
2   throw new Error('[!] Nome non valido');
```

Il percorso di disambiguazione degli omonimi mai esercitato

La stesura delle verifiche per la creazione degli utenti ha mostrato che il ciclo di generazione dei progressivi non era mai stato attivato durante tutto il processo di sviluppo. Questo è dovuto alla natura del meccanismo, che si attiva solo se l'inserimento di un utente viola il vincolo di unicità sull'indirizzo email, il che implica che ci debba essere un utente con lo stesso indirizzo. Non si può parlare di un difetto, ma di una parte della logica che non era stata verificata.

La verifica automatica ha permesso di simulare il fallimento del primo inserimento, offrendo così una protezione contro possibili regressioni in futuro.

Considerazioni

L'attività di verifica ha dimostrato quanto sia utile seguire criteri sistematici piuttosto che scegliere i casi da testare in modo casuale. Inoltre, è importante notare che l'adozione dell'iniezione delle dipendenze, giustificata nel terzo capitolo per motivi di disaccoppiamento, si è rivelata fondamentale per garantire la verificabilità. Senza questa pratica, i metodi dei servizi non sarebbero stati utilizzabili senza una base di dati, e la verifica avrebbe dovuto avvenire a livello di sistema, con costi di esecuzione così elevati da rendere difficile la ripetizione.

5.2 Valutazione dell'usabilità

5.2.1 Expert review

L'ispezione euristica è un metodo di valutazione dell'usabilità che viene effettuato da uno o più esaminatori. Questi analizzano l'interfaccia confrontandola con un insieme di principi ben definiti, piuttosto che osservare come la utilizzano gli utenti reali. Questo approccio, ideato da Jakob Nielsen e Rolf Molich, ha il vantaggio di non richiedere la partecipazione di utenti esterni, il che significa che può essere applicato in qualsiasi fase di sviluppo.

L'ispezione è stata effettuata prima di coinvolgere gli utenti. Infatti, le problematiche che una revisione sistematica può identificare sarebbero emerse anche durante l'osservazione dei partecipanti, ma con la differenza che avrebbero occupato il tempo delle sessioni. Presentare agli utenti un prodotto già corretto nei suoi difetti più evidenti permette di concentrare l'osservazione su aspetti che solo l'uso reale può far emergere.

Il metodo adottato

N	Euristica	Enunciato
1	Visibilità dello stato del sistema	Il sistema deve mantenere l'utente informato su ciò che sta accadendo, mediante riscontri adeguati e tempestivi
2	Corrispondenza tra sistema e mondo reale	Il linguaggio impiegato deve essere quello dell'utente, non quello del sistema
3	Controllo e libertà	L'utente deve poter annullare le operazioni intraprese per errore
4	Coerenza e standard	Elementi analoghi devono comportarsi in modo analogo, e le convenzioni consolidate devono essere rispettate
5	Prevenzione dell'errore	È preferibile impedire l'errore anziché segnalarlo a posteriori

6	Riconoscere anziché ricordare	Le informazioni necessarie devono essere visibili, non dover essere memorizzate
7	Flessibilità ed efficienza d'uso	Il sistema deve risultare accessibile all'utente inesperto e rapido per quello esperto
8	Design essenziale	Le informazioni superflue competono con quelle rilevanti
9	Riconoscimento e recupero degli errori	I messaggi devono indicare il problema in linguaggio comprensibile e suggerire una soluzione
10	Guida e documentazione	Ove necessaria, deve essere reperibile e circoscritta al compito da svolgere

L'ispezione è stata effettuata in due fasi consecutive. Nella prima, ogni schermata è stata analizzata in base a tutte le euristiche; nella seconda, è stato rivisitato l'intera applicazione, concentrandoci su un solo principio alla volta. Questo approccio ci ha permesso di individuare incoerenze tra diverse viste, che potrebbero sfuggire se si esamina una schermata in modo isolato.

A ogni difformità riscontrata è stata assegnata una valutazione di gravità, seguendo la scala proposta dallo stesso autore. Questa scala tiene conto della frequenza con cui il problema si presenta, dell'entità dell'ostacolo per l'utente e della sua persistenza, ovvero quanto il problema continui a manifestarsi anche dopo che l'utente ha preso confidenza con il sistema.

Livello	Denominazione	Criterio
0	Non è un problema	La circostanza rilevata non manomette l'uso
1	Cosmetico	Da correggere se si ha tempo residuo
2	Minore	Ostacola l'uso ma può essere aggirato
3	Maggiore	Compromette il conseguimento dell'obiettivo e persiste con l'uso
4	Catastrofico	Impedisce l'utilizzo del sistema

Esiti dell'ispezione

N	Schermata	Euristica	Descrizione	Severità	Correzione
1	Accesso	9	Quando vengono inserite credenziali non valide, il sistema mostra il messaggio "Impossibile contattare il server" invece del messaggio corretto "Credenziali non valide". L'utente è quindi portato ad aspettare che il server riprenda l'esecuzione correttamente o ad aggiornare la pagina continuamente, invece di controllare le proprie credenziali	3	Controllare che il messaggio di risposta nel codice sia coerente con il problema riscontrato
2	Dettaglio segnalazione	1	Il messaggio conseguente all'inserimento di una data di scadenza non valida permane anche dopo che l'utente ha corretto il valore, inducendo a ritenere che l'operazione continui a non riuscire	2	Azzerare il messaggio all'atto di ciascun nuovo tentativo

3	Creazione utenti	1	L'anteprima dell'indirizzo email non mostra il numero progressivo di disambiguazione che il sistema aggiungerà in caso di omonimia. Di conseguenza, l'amministratore potrebbe aspettarsi un indirizzo diverso da quello che verrà realmente creato	2	Indicare il progressivo nell'anteprima dell'email
4	Nuova segnalazione, dettaglio segnalazione	9	I messaggi mostrano il prefisso [!] e usano frasi come «dati non validi», ma non specificano quale campo sia problematico né quale azione l'utente dovrebbe intraprendere	2	Rimuovere la notazione tecnica e specificare il campo interessato

Anomalie funzionali rilevate incidentalmente

L'ispezione ha portato anche alla scoperta di anomalie di tipo funzionale, che non rientrano esattamente nella valutazione euristica, ma sono comunque un risultato collaterale di essa. È importante menzionarle, poiché la loro identificazione rappresenta un esito dell'attività svolta, anche se non è direttamente legata ai principi applicati.

N	Schermata	Descrizione	Severità	Esito
5	Lista segnalazioni	Il filtro per priorità non produce alcun effetto sui risultati presentati	3	Corretta
6	Creazione utenti	Non è previsto alcun comando per la creazione di utenze amministrative	3	Corretta

5.2.2 Test con utenti reali

L'ispezione euristica di cui abbiamo discusso in precedenza ha un limite: nonostante verifichi se l'interfaccia segue i principi riconosciuti, non può prevedere come un utente reale si comporterà realmente. Solo attraverso l'osservazione diretta possiamo distinguere le discrepanze che hanno un impatto da quelle che rimangono puramente teoriche, e identificare difficoltà che nessun principio avrebbe potuto prevedere. Per questo motivo è stato effettuato un test con utenti, in cui a ciascun partecipante è stato chiesto di completare una serie di compiti senza ricevere istruzioni su come farlo, mentre l'osservatore registrava il loro comportamento senza intervenire.

Il test aveva l'obiettivo di esplorare tre aspetti distinti. Per prima cosa, il capire se un nuovo utente riuscisse a capire cosa fosse una segnalazione, quali tipologie esistessero e cosa significherebbero i vari stati di avanzamento. Poi, il sapere se i comandi necessari per ogni operazione fossero facilmente individuabili senza bisogno di una guida. Infine, il verificare se l'utente fosse in grado di capire, al termine di ogni azione, se avesse avuto successo.

Per condurlo è stata utilizzata la tecnica del *think aloud*, in cui ai partecipanti è stato chiesto di verbalizzare ciò che stavano cercando, le loro aspettative e le motivazioni dietro le loro scelte. L'osservatore ha evitato di intervenire durante il test, anche di fronte a evidenti difficoltà, a meno che il partecipante non rimanesse bloccato per un tempo eccessivo. Questa scelta, sebbene controintuitiva, è stata fondamentale per garantire la validità del test. Prima di iniziare, a ciascun partecipante è stato chiarito che l'oggetto della valutazione era il sistema e non la loro persona, e che ogni difficoltà riscontrata sarebbe stata attribuita alla progettazione dell'interfaccia.

I compiti sono stati pensati come situazioni da risolvere, piuttosto che semplici sequenze di operazioni da seguire. Questa distinzione è fondamentale: l'istruzione «inserisci una nuova segnalazione premendo il comando in alto a destra» testerebbe solo la capacità di seguire istruzioni, mentre la richiesta «segnala il problema che hai

riscontrato» verifica se l'utente è in grado di identificare autonomamente il percorso da seguire.

Dopo aver sottoposto i partecipanti a questa prima fase di verifica, sono state effettuate altre due fasi. La prima prevede la somministrazione di un questionario standardizzato, la System Usability Scale ideata da John Brooke. Questo strumento è composto da dieci affermazioni, per le quali i partecipanti devono esprimere il loro grado di accordo su una scala di cinque livelli, restituendo un punteggio che va da zero a cento. Il secondo momento consiste in un colloquio conclusivo, condotto mediante tre domande aperte:

1. Quale è stata l'operazione che le è risultata più difficile, e per quale ragione?
2. C'è qualcosa che si aspettava di trovare e che non ha trovato?
3. Qualora dovesse impiegare quotidianamente questo strumento, cosa modificherebbe per prima cosa?

Le domande sono state pensate in modo da non suggerire risposte specifiche e da non dare per scontato che ci sia un problema. Il questionario è stato somministrato prima del colloquio. L'ordine delle domande è importante: le domande aperte spingono a riflettere sulle difficoltà riscontrate e potrebbero quindi influenzare le risposte al questionario se venissero poste prima.

N	Compito	Obiettivo della verifica
1	Accedere al sistema per la prima volta	Comprensione dell'obbligo di sostituzione delle credenziali temporanee
2	Segnalare un malfunzionamento riscontrato	Individuare il percorso di inserimento e criteri di classificazione adottati
3	Verificare lo stato di una segnalazione altrui	Impiego degli strumenti di ricerca e comprensione degli stati
4	Contribuire alla discussione su una segnalazione	Riconoscimento della sezione dei commenti

5	Aggiornare lo stato di una segnalazione affidata	Individuazione del comando e comprensione delle conseguenze
6	Creare un account e affidarvi una segnalazione	Comprensione della generazione automatica dell'indirizzo e dei comandi amministrativi

Il compito 5 è stato somministrato ai soli partecipanti cui era stato attribuito il ruolo di assegnatario, il compito 6 ai soli partecipanti con profilo amministrativo.

Partecipanti e risultati ottenuti

Profilo	Studente universitario in matematica, si dedica alla programmazione come interesse personale
Competenza tecnica	Media
Esperienza con strumenti analoghi	Limitata
Ruolo assegnato	Utente ordinario

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato con difficoltà	0	1
3 - Ritrovare una segnalazione	Completato	0	0
4 - Commentare	Completato	0	0
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Non applicabile	-	-

Nel corso del compito 2 il partecipante ha portato a termine correttamente l'inserimento, ma non ha riconosciuto che l'operazione fosse riuscita. Al termine dell'invio si è trovato sulla schermata di dettaglio della segnalazione appena creata senza percepire che questa costituisse la conferma dell'avvenuta creazione, e ha richiesto conferma verbale.

Per il calcolo del **punteggio SUS** è stato effettuato il seguente calcolo:

Dispari(risposta - 1) : $5 - 1 = 4, 5 - 1 = 4, 5 - 1 = 4, 5 - 1 = 4, 2 - 1 = 1 \rightarrow 17$

Pari(5 - risposta) : $5 - 1 = 4, 5 - 1 = 4, 5 - 1 = 4, 5 - 1 = 4, 5 - 1 = 4 \rightarrow 20$

Il calcolo finale sarebbe:

$$\text{Somma } 372,5 = 92,5$$

Per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: nessuna in particolare. Ha trovato il sistema piuttosto intuitivo e ha apprezzato il fatto che le operazioni non consentite per il suo ruolo non vengano semplicemente disabilitate, ma non vengano nemmeno mostrate.
- Aspettative disattese: la possibilità di allegare un'immagine ai commenti, una funzionalità che si aspettava di trovare, considerando che era simile a quella disponibile per le segnalazioni.
- Prima modifica che apporterebbe: vorrebbe vedere l'introduzione di messaggi di conferma dopo aver creato o modificato una segnalazione.

Profilo	Studentessa universitaria in informatica
Competenza tecnica	Elevata
Esperienza con strumenti analoghi	Limitata
Ruolo assegnato	Utente ordinario

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato con difficoltà	0	1
3 - Ritrovare una segnalazione	Completato	0	0
4 - Commentare	Completato con difficoltà	0	1
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Non applicabile	-	-

La partecipante ha frainteso il comando "Invia", situato alla fine del modulo dei commenti, pensando che questo confermasse sia l'inserimento del commento che il caricamento dell'immagine. La vicinanza visiva tra l'area di caricamento dell'allegato e la sezione della discussione, entrambe presenti nella schermata di dettaglio, l'ha portata a considerarle come un unico modulo invece che come elementi separati.

Il **punteggio SUS** è pari a 97.5. Mentre per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: nessuna in particolare.
- Aspettative disattese: nessuna in particolare.
- Prima modifica che apporterebbe: introduzione di un filtro per assegnatario nell'elenco delle segnalazioni, in modo da poter restringere la visualizzazione a quelle assegnate a un utente specifico.

Profilo	Studente universitario in chimica, utilizzatore assiduo di strumenti informatici
Competenza tecnica	Media
Esperienza con strumenti analoghi	Nessuna
Ruolo assegnato	Amministratore

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato	0	0
3 - Ritrovare una segnalazione	Completato	0	0
4 - Commentare	Completato	0	0
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Completato	0	0

È il primo partecipante che ha completato tutti i compiti senza alcuna difficoltà, incluso quello amministrativo. Questo è particolarmente interessante, considerando che il partecipante non aveva mai utilizzato strumenti simili prima, e dimostra che la facilità d'uso del sistema non è legata all'esperienza pregressa nel settore.

Il **punteggio SUS** è pari a 95. Mentre per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: nessuna in particolare.
- Aspettative disattese: una procedura di recupero della password in caso di smarrimento.
- Prima modifica che apporterebbe: differenziazione più marcata tra le segnalazioni di priorità critica e quelle di priorità alta, le cui rappresentazioni grafiche ha giudicato eccessivamente simili. Ha inoltre suggerito l'introduzione di un filtro per assegnatario.

Profilo	Studente universitario in informatica, diplomato presso istituto tecnico informatico
Competenza tecnica	Elevata
Esperienza con strumenti analoghi	Consolidata
Ruolo assegnato	Amministratore

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato	0	0
3 - Ritrovare una segnalazione	Completato	0	0
4 - Commentare	Completato	0	0
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Completato	0	0

Anche se il partecipante ha completato tutti i compiti senza problemi, ha espresso qualche dubbio riguardo a due operazioni. Per quanto riguarda il caricamento dell'immagine, il tempo che è passato tra la selezione del file e la sua apparizione — causato dall'elaborazione e dalla trasmissione — è stato visto come un possibile malfunzionamento, soprattutto perché non c'era alcuna indicazione sullo stato di avanzamento. Per quanto riguarda la creazione di una segnalazione, non ha riconosciuto la schermata di dettaglio come conferma che la registrazione fosse avvenuta con successo.

Il **punteggio SUS** è pari a 92.5. Mentre per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: nessuna in particolare.
- Aspettative disattese: È importante avere un riscontro visivo mentre l'immagine si carica, insieme a una conferma chiara che la segnalazione è stata creata con successo. Inoltre, è stata notata una discrepanza nei termini: la voce di navigazione è chiamata "Issue", mentre il titolo della schermata corrispondente è "Segnalazioni".
- Prima modifica che apporterebbe: l'introduzione di un riscontro durante il caricamento dell'immagine.
- Ulteriori suggerimenti: l'aggiunta, nella vista di sintesi, di un collegamento all'elenco completo degli utenti; la predisposizione di una schermata dedicata alle informazioni del proprio profilo.

Profilo	Laureata in biotecnologie della salute
Competenza tecnica	Media
Esperienza con strumenti analoghi	Limitata, ha impiegato un sistema di gestione ticket per l'assistenza informatica
Ruolo assegnato	Utente ordinario

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato	0	0
3 - Ritrovare una segnalazione	Completato	0	0
4 - Commentare	Completato	0	0
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Non applicabile	-	-

È l'unica partecipante ad avere un po' di esperienza con sistema di gestione delle segnalazioni, anche se come utilizzatrice occasionale.

Il **punteggio SUS** è pari a 100. Mentre per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: nessuna in particolare.
- Aspettative disattese: un messaggio di conferma successivo all'aggiornamento dello stato di una segnalazione.
- Prima modifica che apporterebbe: nessuna in particolare.

Profilo	Laureata in sociologia
Competenza tecnica	Bassa
Esperienza con strumenti analoghi	Nessuna
Ruolo assegnato	Utente ordinario

Compito	Esito	Errori	Aiuti
1 - Primo accesso	Completato	0	0
2 - Segnalare un problema	Completato	0	0
3 - Ritrovare una segnalazione	Completato con difficoltà	1	1
4 - Commentare	Completato	0	0
5 - Aggiornare lo stato	Completato	0	0
6 - Amministrazione	Non applicabile	-	-

Durante il compito 3, quando si trattava di identificare chi dovesse risolvere il problema, la partecipante ha commesso l'errore di indicare il segnalatore invece dell'assegnatario. Infatti, nell'elenco delle segnalazioni, il nome dell'autore è scritto accanto alla data, mentre l'assegnatario è rappresentato solo da un disco che le iniziali, posizionato all'estremità destra della riga. Questa differenza nel modo di rappresentarli, ha portato a confondere il primo con il secondo.

Il **punteggio SUS** è pari a 87,5. Mentre per quanto riguarda le risposte alle domande conclusive:

- Operazione più difficile: individuazione dell'assegnatario.

- Aspettative disattese: un messaggio di conferma di avvenuta creazione della segnalazione. Ha inoltre riferito che l'aver messo l'inserimento dell'immagine dopo la creazione, induce a dubitare che la procedura di creazione sia conclusa.
- Prima modifica che apporterebbe: nessuna in particolare.

Sintesi dei risultati

	Profilo	Competenza	Esperienza pregressa	Ruolo
P1	Studente in matematica	Media	Limitata	Utente
P2	Studentessa in informatica	Elevata	Limitata	Utente
P3	Studente in chimica	Media	Nessuna	Amministratore
P4	Studente in informatica	Elevata	Consolidata	Amministratore
P5	Laureata in biotecnologie	Media	Limitata	Utente
P6	Laureata in sociologia	Bassa	Nessuna	Utente

La composizione mostra chiaramente come si distribuiscono i vari profili emersi durante l'analisi: due partecipanti si allineano all'archetipo del membro del team di sviluppo, altri due rappresentano il responsabile con compiti amministrativi, mentre gli ultimi due sono utilizzatori senza competenze informatiche specifiche, riconducibili al profilo del soggetto esterno.

Esiti dei compiti

Compito	Completati	Con difficoltà	Non completati	Tasso di successo
1 - Primo accesso	6	0	0	100%

2 - Segnalare un problema	4	2	0	100%
3 - Ritrovare una segnalazione	5	1	0	100%
4 - Commentare	5	1	0	100%
5 - Aggiornare lo stato	6	0	0	100%
6 - Amministrazione	2	0	0	100%

Tutti i partecipanti hanno completato con successo i compiti assegnati. Solo quattro esecuzioni su trenta hanno mostrato qualche difficoltà e nell'intero test è stato commesso un solo errore. Sebbene i risultati siano positivi, è importante precisare che l'assenza di fallimenti non significa che non ci siano stati problemi. Come vedremo, la maggior parte delle difficoltà riscontrate riguarda la percezione del risultato piuttosto che la capacità di raggiungerlo.

Punteggi della System Usability Scale

Partecipante	Punteggio
P1	92.5
P2	97.5
P3	95.0
P4	92.5
P5	100
P6	87.5
Media	94.2

Criticità emerse

Il test ha condotto all'individuazione delle criticità seguenti, ordinate secondo il numero di partecipanti che le hanno manifestate.

N	Criticità	Partecipanti	Euristica	Severità
1	Assenza di conferma esplicita successiva alla creazione di una segnalazione, all'aggiornamento dello stato e al caricamento di un'immagine	P1, P4, P5, P6	1 - Visibilità dello stato	2
2	Rapporto poco chiaro tra l'area di caricamento dell'immagine e il modulo dei commenti, collocati in prossimità nella schermata di dettaglio	P2, P6	4 - Coerenza e 8 - Design essenziale	2
3	Assenza di un filtro per assegnatario nell'elenco delle segnalazioni	P2, P3	7 - Flessibilità ed efficienza	2
4	Rappresentazione grafica dell'assegnatario meno evidente rispetto a quello dell'autore	P6	2 - Corrispondenza col mondo reale	3
5	Insufficiente differenziazione cromatica tra le priorità alta e critica	P3	4 - Coerenza e standard	2

6	Incoerenza terminologica tra la voce di navigazione "Issue" e il titolo della schermata "Segnalazioni"	P4	4 - Coerenza e standard	1
7	Assenza di riscontro durante il caricamento dell'immagine	P4	1 - Visibilità dello stato	2
8	Impossibilità di associare un'immagine a un commento	P1, P2	Funzionalità non prevista	1

Conclusioni

Il test ha confermato che l'interfaccia è complessivamente adeguata, ma ha anche messo in luce una mancanza sistematica che l'ispezione euristica non aveva colto: non c'è nessun meccanismo generale per fornire un riscontro dopo le operazioni riuscite. Questa situazione merita attenzione.

Infine, si nota una corrispondenza tra le previsioni fatte durante la caratterizzazione degli utenti e quanto osservato sul campo.